

ระบบรับส่งรูปภาพไร้สายระยะไกลด้วย LoRa

Long-Range Wireless Image Transmission System using LoRa

จัดทำโดย

นาย ณัฐสิทธิ์ สมสุข	รหัสนักศึกษา 67200087
นาย ธีรดนย์ ทองเทพ	รหัสนักศึกษา 67200128
นาย พีระนันท์ ปราชญ์เปรื่อง	รหัสนักศึกษา 67200205
นายวรรลภย์ ศรีเบญจรัตน์	รหัสนักศึกษา 67200244
นายวริทธิ์ธร คำภา	รหัสนักศึกษา 67200245
นาย พาวงษ์ ศรสิทธิ์	รหัสนักศึกษา 67200391

รายงานนี้เป็นส่วนหนึ่งของวิชา WIRELESS COMMUNICATION

หลักสูตรวิศวกรรมศาสตร์บัณฑิต สาขาวิชาวิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์

อัจฉริยะ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

วิทยาเขตชุมพรเขตอุดมศักดิ์จังหวัดชุมพร

ปีการศึกษา 2569

รายงานปีการศึกษา 2569

สาขาวิชาวิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์อัจฉริยะ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง วิทยาเขตชุมพรเขตอุดมศักดิ์ จังหวัดชุมพร

เรื่อง ระบบรับส่งรูปภาพไร้สายระยะไกลด้วย LoRa

Long-Range Wireless Image Transmission System using LoRa

ผู้จัดทำ

นาย ณัฐสิทธิ์ สมสุข	67200087
นาย ธีรดนย์ ทองเทพ	67200128
นาย พีระนันท์ ปราชญ์เปรื่อง	67200205
นาย วรรลภย์ ศรีเบญจรัตน์	67200244
นาย วริทธิ์ธร คำภา	67200245
นาย พาวงษ์ ศรสิทธิ์	67200391

อาจารย์ที่ปรึกษา

(อ. อรวรรณ หนูอิ่ม)

(.....)

ชื่อรายงาน ระบบรับส่งรูปภาพไร้สายระยะไกลด้วย LoRa

ชื่อผู้จัดทำ นาย ณัฐสิทธิ์ สมสุข รหัสนักศึกษา 67200087
นาย ธีรดนย์ ทองเทพ รหัสนักศึกษา 67200128
นาย พีระนันท์ ปราชญ์เปรื่อง รหัสนักศึกษา 67200205
นายวรรลภย์ ศรีเบญจรัตน์ รหัสนักศึกษา 67200244
นายวริทธิ์ธร คำภา รหัสนักศึกษา 67200245
นาย พาวงษ์ ศรสิทธิ์ รหัสนักศึกษา 67200391

อาจารย์ที่ปรึกษา อ. อรวรรณ หนูอิ่ม

หลักสูตร วิศวกรรมศาสตร์บัณฑิต

สาขาวิชา วิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์อัจฉริยะ

ปีการศึกษา 2569

บทคัดย่อ

งานวิจัยและพัฒนานี้มีวัตถุประสงค์เพื่อออกแบบและสร้างระบบสะพานเชื่อมต่อข้อมูลอนุกรมไร้สาย (LoRa Serial Bridge) สำหรับการส่งไฟล์รูปภาพระหว่างคอมพิวเตอร์สองเครื่องผ่านคลื่นความถี่วิทยุระยะไกล โดยอาศัยบอร์ดไมโครคอนโทรลเลอร์ ESP32 ทำหน้าที่เป็นตัวกลางเชื่อมต่อกับโมดูล REYAX RYLR998 LoRa ผ่านพอร์ตสื่อสารอนุกรม (UART)

เนื่องจากโมดูล RYLR998 มีข้อจำกัดด้านขนาดข้อมูลสูงสุดต่อแพ็กเก็ต (Maximum Payload Length 240 ไบต์) และส่งได้เฉพาะข้อมูลข้อความ ระบบจึงได้รับการออกแบบกระบวนการประมวลผลข้อมูลภาพด้วยภาษา Python บนคอมพิวเตอร์ต้นทาง เริ่มจากการปรับขนาดรูปภาพและบีบอัดไฟล์ให้อยู่ในรูปแบบ JPEG จากนั้นแปลงข้อมูลไบนารีเป็นข้อความด้วยรหัส Base64 แล้วทำการแบ่งข้อมูลออกเป็นส่วนๆ (Chunking) เพื่อส่งผ่านพอร์ต USB ไปยังบอร์ด ESP32 ฝั่งส่ง ซึ่งจะนำข้อมูลมาจัดรูปแบบเป็นคำสั่ง AT Command (AT+SEND) เพื่อกระจายสัญญาณไร้สายผ่านโมดูล LoRa สำหรับ ESP32 ฝั่งรับจะคอยตรวจจับสัญญาณการรับข้อมูล (+RCV) สกัดเอาเฉพาะส่วน Payload แล้วส่งต่อผ่าน USB ไปยังคอมพิวเตอร์ปลายทาง เพื่อรวมรวมแพ็กเก็ตตรวจสอบความถูกต้อง ถอดรหัส Base64 และบันทึกกลับเป็นไฟล์รูปภาพฉบับสมบูรณ์

ผลการทดสอบระบบพบว่า โครงสร้างโปรโตคอลสื่อสารที่พัฒนาขึ้นสามารถส่งผ่านข้อมูลรูปภาพระหว่างคอมพิวเตอร์ข้ามฝั่งได้สำเร็จโดยไม่มีปัญหาข้อมูลสูญหายหรือผิดพลาด การบีบอัดภาพและการใช้ Base64 Chunking ร่วมกับ LoRa Serial Bridge ช่วยก้าวข้ามข้อจำกัดด้าน Bandwidth และ Payload ของเทคโนโลยี LoRa ได้อย่างมีประสิทธิภาพ เหมาะสำหรับการนำไปประยุกต์ใช้งานในพื้นที่ห่างไกลที่ไม่สัญญาณอินเทอร์เน็ต เช่น การส่งภาพถ่ายจากกล้องตรวจการณ์พื้นที่เกษตรกรรมแม่นยำ หรือการสำรวจภัยพิบัติทางธรรมชาติ

Project Title Long-Range Wireless Image Transmission System using LoRa

Degree Bachelor of Engineering.

Program in Robotics and Intelligent Electronics Engineering.

Academic Year 2026

ABSTRACT

This project presents the design and implementation of a LoRa Serial Bridge system for long-range wireless image file transmission between two computers. The system utilizes ESP32 microcontrollers as interfacing bridges connected to REYAX RYLR998 LoRa modules via Universal Asynchronous Receiver-Transmitter (UART) serial communication.

Due to the constraints of the RYLR998 module, including a maximum payload size of 240 bytes per packet and text-only AT command transmission, a dedicated data processing pipeline was developed using Python on the transmitting computer. The pipeline resizes and compresses image files into JPEG format, encodes the binary data into Base64 strings, and segments the encoded data into smaller chunks. These chunks are sequentially forwarded via USB Serial to the transmitting ESP32, which encapsulates each chunk into an AT command (AT+SEND) for wireless transmission. On the receiving end, the receiving ESP32 captures incoming data packets (+RCV), extracts the payload, and forwards it to the receiving computer via USB Serial. The destination software reconstructs the packets, verifies completeness, decodes the Base64 data, and saves the received output as a fully restored image file.

Experimental results demonstrate that the proposed system achieves reliable inter-computer wireless image transmission without data corruption. The combination of image compression, Base64 chunking, and the LoRa Serial Bridge successfully overcomes the throughput and payload limitations inherent to LoRa technology. This system provides a practical solution for remote monitoring applications lacking internet connectivity, such as environmental surveillance, precision agriculture, and disaster response.

กิตติกรรมประกาศ

โครงงาน "ระบบรับส่งรูปภาพไร้สายระยะไกลด้วย LoRa" (Long-Range Wireless Image Transmission System using LoRa) ฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีด้วยความกรุณาและความช่วยเหลืออย่างดียิ่งจากอาจารย์ที่ปรึกษาโครงงาน [ชื่อ-นามสกุล อาจารย์ที่ปรึกษา] ที่ได้ให้คำแนะนำ แนวคิด คำปรึกษา ตลอดจนช่วยตรวจทานและแก้ไขข้อบกพร่องต่างๆ ในการออกแบบโครงสร้างระบบและการเขียนโปรแกรมควบคุมอย่างดียิ่ง คณะผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูง ณ โอกาสนี้

ขอขอบพระคุณคณะครู/อาจารย์ประจำสาขาวิชา [ชื่อสาขาวิชา] ทุกท่าน ที่ได้ประสิทธิ์ประศาสน์วิชาความรู้ทั้งภาคทฤษฎีและภาคปฏิบัติในด้านไมโครคอนโทรลเลอร์ ระบบการสื่อสารไร้สาย และการเขียนโปรแกรม ซึ่งเป็นพื้นฐานสำคัญยิ่งในการนำมาประยุกต์ใช้ในการจัดทำโครงงานนี้

ขอกราบขอบพระคุณบิดา มารดา และผู้ปกครองทุกท่าน ที่ได้ให้การสนับสนุนทั้งด้านทุนทรัพย์ กำลังใจ และความช่วยเหลือในทุกๆ ด้านอย่างเต็มที่เสมอมา รวมถึงขอขอบคุณเพื่อนๆ ร่วมชั้นเรียนทุกคนที่ได้ให้คำแนะนำ แลกเปลี่ยนแนวคิด และให้กำลังใจซึ่งกันและกันตลอดการดำเนินงานโครงงาน

คุณค่าและประโยชน์อันพึงมีจากโครงงานฉบับนี้ คณะผู้จัดทำขอมอบเป็นกตเวทิตาคุณแด่ผู้มีพระคุณทุกท่านที่มีส่วนช่วยให้โครงงานนี้สำเร็จลุล่วงได้ด้วยดี

ณัฐสิทธิ์ สมสุข
ธีรดนย์ ทองเทพ
พีระนันท์ ปราชญ์เปรื่อง
วรรลภย์ ศรีเบญจรัตน์
วริทธิ์ธร คำภา
พาวงษ์ ศรสิทธิ์

สารบัญ

เรื่อง	หน้า
บทคัดย่อภาษาไทย	ก
บทคัดย่อภาษาอังกฤษ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง-ช
สารบัญตาราง	ซ
สารบัญรูป	ฌ
บทที่ 1 บทนำ	**1**
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงงาน	2
1.3 ขอบเขตของโครงงาน	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	4
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	**5**
2.1 เทคโนโลยีการสื่อสารไร้สาย LoRa และโปรโตคอล LoRaWAN	5
2.1.1 เทคโนโลยี LoRa	5
2.1.2 ค่าพารามิเตอร์ทางกายภาพของ LoRa	6
2.1.3 ความแตกต่างระหว่าง LoRa และ LoRaWAN	7
2.2 คุณลักษณะทางเทคนิคของโมดูล REYAX RYLR998	8
2.2.1 คุณสมบัติทางไฟฟ้าและประสิทธิภาพคลื่นวิทยุ	8
2.2.2 รูปแบบโครงสร้างชุดคำสั่ง AT Command ที่สำคัญ	9
2.2.3 รูปแบบโครงสร้างแพ็กเกจการรับข้อมูล	9
2.3 ไมโครคอนโทรลเลอร์ ESP32 และการสื่อสารอนุกรม	10
2.3.1 คุณลักษณะของไมโครคอนโทรลเลอร์ ESP32	10
2.3.2 การสื่อสารอนุกรมแบบอนุกรมระดับฮาร์ดแวร์	10
2.3.3 หลักการทำงานแบบบริดจ์ส่งผ่านข้อมูล	11
2.3.4 การตั้งค่าอัตราเร็วสื่อสารและการขยายบัฟเฟอร์ข้อมูล	12

สารบัญ(ต่อ)

เรื่อง	หน้า
2.4 การบีบอัดไฟล์ภาพ JPEG และการแปลงรหัส	12
2.4.1 การบีบอัดและปรับขนาดภาพ JPEG	12
2.4.2 การย่อยชิ้นส่วนข้อมูล	13
2.4.3 รูปแบบการแปลงรหัสข้อมูล	13
2.5 เทคนิคการแบ่งแพ็กเก็ตข้อมูล	14
2.5.1 โครงสร้างแพ็กเก็ตข้อมูลและส่วนหัว	14
2.5.2 ขั้นตอนการซอยย่อยข้อมูล	14
2.5.3 การจัดการเวลาหน่วงแอร์ไทม์	15
2.6 งานวิจัยและเอกสารที่เกี่ยวข้อง	15
2.6.1 การประยุกต์ใช้เทคโนโลยี LoRa สำหรับการรับส่งข้อมูลรูปภาพ	15
2.6.2 การพัฒนากลไกโปรโตคอลตรวจสอบความถูกต้องและส่งซ่อมข้อมูล	16
2.6.3 การศึกษาผลกระทบพารามิเตอร์ Spreading Factor และแอร์ไทม์	16
2.6.4 การประยุกต์ใช้ไมโครคอนโทรลเลอร์เป็น	16
บทที่ 3 การออกแบบและพัฒนาระบบ	**17**
3.1 สถาปัตยกรรมรวมของระบบ	17
3.1.1 การแบ่งระดับชั้นการทำงาน ระบบแบ่งออกเป็น 3 ระดับชั้นหลักทั้งฝั่งส่ง	17
3.1.2 ไดอะแกรมการเชื่อมต่อสัญญาณระหว่างส่วนประกอบ	18
3.1.3 รายการอุปกรณ์ฮาร์ดแวร์ในระบบ	19
3.1.4 ผังการต่อสายสัญญาณ	20
3.1.5 รายละเอียดและข้อพิจารณาทางเทคนิคในการต่อวงจร	20
3.2 การออกแบบโปรโตคอลการสื่อสารข้อมูล	21
3.2.1 รูปแบบและโครงสร้างแพ็กเก็ต	21
3.2.2 ลำดับขั้นตอนการทำงานของโปรโตคอล	22
3.2.3 กลไกการทำงานของโปรโตคอลในแต่ละเฟส	22
3.3 การพัฒนาโปรแกรมประมวลผลและส่งภาพบน Computer A	23
3.3.1 โครงสร้างซอฟต์แวร์และไลบรารีที่ใช้งาน	24

สารบัญ(ต่อ)

เรื่อง	หน้า
3.3.2 อัลกอริทึมและขั้นตอนการทำงานของโปรแกรม	24
3.3.3 ซอร์สโค้ดโปรแกรมประมวลผลฝั่งส่ง	25
3.4 การพัฒนาโปรแกรมไมโครคอนโทรลเลอร์ ESP32 #1	29
3.4.1 หน้าที่และโลจิกการทำงานของเฟิร์มแวร์	29
3.4.2 การตั้งค่าความเร็วและการจัดการบัฟเฟอร์	29
3.4.3 ซอร์สโค้ดโปรแกรม ESP32 #1	30
3.5 การพัฒนาโปรแกรมรับและประมวลผลภาพบน Computer B	31
3.5.2 อัลกอริทึมและขั้นตอนการทำงานของโปรแกรม	31
3.5.3 ซอร์สโค้ดโปรแกรมประมวลผลฝั่งรับ	32
บทที่ 4 ผลการทดสอบและการวิเคราะห์ผล	**36**
4.1 ตารางสรุปผลการทดสอบประสิทธิภาพการสื่อสาร	36
4.2 การวิเคราะห์ผลการทดสอบ	37
4.2.2 วิเคราะห์อัตราการรับส่งแพ็กเกจสำเร็จ	37
4.3 บทบาทและความสำคัญของโปรโตคอล ARQ ต่อความสมบูรณ์ของรูปภาพ	38
บทที่ 5 สรุปผล ข้อเสนอแนะ และแนวทางการพัฒนาต่อยอด	**39**
5.1 สรุปผลการดำเนินงาน	39
5.1.1 สรุปด้านสถาปัตยกรรมและการพัฒนาฮาร์ดแวร์	39
5.1.2 สรุปด้านโปรโตคอลการสื่อสารและการจัดการไฟล์ภาพ	39
5.1.3 สรุปผลการทดสอบประสิทธิภาพการสื่อสารไร้สาย	40

สารบัญ(ต่อ)

เรื่อง	หน้า
5.2 ข้อเสนอแนะ	40
5.2.1 ด้านการปรับปรุงโปรโตคอลการสื่อสารและซอฟต์แวร์	40
5.2.2 ด้านฮาร์ดแวร์และการแพร่กระจายคลื่นวิทยุ	41
5.2.3 ด้านการนำไปประยุกต์ใช้งานจริง	41
5.3 แนวทางการพัฒนาต่อยอด	41
5.3.1 การพัฒนาเป็นระบบโหนดเครือข่ายไร้สาย	41
5.3.2 การย้ายประมวลผลไปยัง Edge Computing และฮาร์ดแวร์ฝังตัว	42
5.3.3 การบริหารจัดการพลังงานและการประยุกต์ใช้พลังงานทางเลือก	42
5.3.4 การพัฒนาส่วนประสานผู้ใช้และการเชื่อมต่อระบบคลาวด์	42
บรรณานุกรม	**43**
ภาคผนวก ก ซอร์สโค้ดเฟิร์มแวร์ ESP32	**44**
ภาคผนวก ข ซอร์สโค้ดเฟิร์มแวร์ python	**46**

สารบัญตาราง

ตารางที่	หน้า
2.1.3 ความแตกต่างระหว่าง LoRa และ LoRaWAN	7
2.2.2 รูปแบบโครงสร้างชุดคำสั่ง AT Command ที่สำคัญ	9
3.2 ตารางการต่อสายสัญญาณ	20
4.1 ตารางสรุปผลการทดสอบประสิทธิภาพการสื่อสาร	36

สารบัญรูป

รูปที่	หน้า
3.1 สถาปัตยกรรมรวมของระบบ (System Architecture)	17
3.1.2 ไดอะแกรมการเชื่อมต่อสัญญาณระหว่างส่วนประกอบ	19
3.3.2 ลำดับขั้นตอนการทำงานของโปรโตคอล	22
4.3 บทบาทและความสำคัญของโปรโตคอล ARQ ต่อความสมบูรณ์ของรูปภาพ	38

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบัน ระบบการสื่อสารไร้สาย (Wireless Communication System) ได้เข้ามามีบทบาทสำคัญอย่างยิ่งต่อการเข้าถึงและการแลกเปลี่ยนข้อมูลในหลากหลายภาคส่วน ไม่ว่าจะเป็นงานด้านการเกษตรแม่นยำ (Precision Agriculture) การเฝ้าระวังสิ่งแวดล้อม การตรวจการณ์ในพื้นที่ห่างไกล หรือการจัดการภัยพิบัติทางธรรมชาติ โดยทั่วไปแล้ว การส่งข้อมูลมัลติมีเดีย เช่น ไฟล์รูปภาพ มักพึ่งพาโครงข่ายสัญญาณโทรศัพท์เคลื่อนที่ (Cellular Networks เช่น 3G/4G/5G) หรือระบบเครือข่ายไร้สายระยะสั้นอย่าง Wi-Fi อย่างไรก็ตาม ในพื้นที่ทุรกันดาร พื้นที่ป่าเขา หรือในสภาวะวิกฤตที่โครงสร้างพื้นฐานด้านการสื่อสารได้รับความเสียหาย สัญญาณเครือข่ายเหล่านี้มักจะไม่สามารถใช้งานได้หรือไม่ครอบคลุมพื้นที่ ส่งผลให้การส่งข้อมูลภาพถ่าย ซึ่งมีความสำคัญต่อการประเมินสถานการณ์และการตัดสินใจในทันที เป็นไปได้ยากลำบากหรือถูกตัดขาดอย่างสิ้นเชิง

เทคโนโลยีการสื่อสารไร้สายแบบ LoRa (Long Range) ซึ่งเป็นเทคโนโลยีการส่งสัญญาณวิทยุในย่านความถี่ไร้ใบอนุญาต (Unlicensed ISM Band) จึงได้กลายมาเป็นทางเลือกที่ได้รับความนิยมสูง เนื่องจากมีจุดเด่นในด้านการส่งสัญญาณระยะไกล (ตั้งแต่หลายกิโลเมตรในพื้นที่เปิดโล่ง) ใช้พลังงานต่ำ และมีความทนทานต่อสัญญาณรบกวน อย่างไรก็ดี LoRa ถูกออกแบบมาสำหรับการส่งข้อมูลขนาดเล็ก เช่น ค่าจากเซนเซอร์ (Sensor Data) จึงมีข้อจำกัดที่สำคัญด้านอัตราการส่งข้อมูล (Data Rate) ที่ต่ำมาก และจำกัดขนาดของข้อความต่อแพ็กเก็ต (Maximum Payload Length) ซึ่งสำหรับโมดูล REYAX RYLR998 นั้น สามารถส่งได้สูงสุดเพียง 240 ไบต์ต่อคำสั่ง AT+SEND เท่านั้น

จากการศึกษาสภาพปัญหาดังกล่าว การส่งไฟล์รูปภาพซึ่งเป็นข้อมูลไบนารี (Binary Data) ขนาดใหญ่ผ่านโมดูล LoRa โดยตรงจึงไม่สามารถทำได้ด้วยวิธีการปกติ จำเป็นต้องมีการออกแบบกระบวนการประมวลผลข้อมูลภาพและการกำหนดโปรโตคอลการรับส่งใหม่ทั้งหมด คณะผู้จัดทำจึงได้เสนอแนวทางการพัฒนาระบบสะพานเชื่อมต่อข้อมูลอนุกรมไร้สาย (LoRa Serial Bridge) โดยใช้บอร์ดไมโครคอนโทรลเลอร์ ESP32 เชื่อมต่อกับโมดูล RYLR998 ผ่านพอร์ต UART และพัฒนาโปรแกรมประมวลผลบนคอมพิวเตอร์เพื่อบีบอัดไฟล์ภาพ แปลงรหัสเป็น Base64 และแบ่งข้อมูลเป็นส่วนๆ (Chunking) ก่อนทำการส่งผ่านคลื่นวิทยุ LoRa ไปยังคอมพิวเตอร์ปลายทาง การศึกษานี้จึงเป็นการก้าวข้ามข้อจำกัดของเทคโนโลยี LoRa เพื่อให้สามารถนำมาประยุกต์ใช้ในการส่งไฟล์ภาพระยะไกลได้อย่างมีประสิทธิภาพและน่าเชื่อถือ

1.2 วัตถุประสงค์ของโครงงาน

จากโครงสร้างและรายละเอียดการทำงานของระบบ LoRa Serial Bridge (923 MHz) วัตถุประสงค์หลักของโครงงานสรุปได้ดังนี้:

1. เพื่อออกแบบและพัฒนาระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกล (LoRa Serial Bridge) เพื่อศึกษากระบวนการเชื่อมต่อและประมวลผลข้อมูลร่วมกันระหว่างคอมพิวเตอร์ ไมโครคอนโทรลเลอร์ ESP32 และโมดูลสื่อสารไร้สาย REYAX RYLR998 บนย่านความถี่ 923 MHz (AS923)
2. เพื่อออกแบบและพัฒนากลไกโปรโตคอลการสื่อสารแบบสองทาง (Two-way Communication) ร่วมกับระบบ ARQ เพื่อสร้างอัลกอริทึมการส่งสัญญาณตอบกลับ (ACK) และระบบตรวจสอบ-ขอส่งซ่อมไฟล์ส่วนที่ขาดหาย (Automatic Repeat Request) สำหรับการส่งผ่านข้อมูล Hex/Base64 เพื่อให้รับ-ส่งไฟล์ภาพได้อย่างถูกต้องแม่นยำ 100% โดยไม่มีข้อมูลสูญหาย
3. เพื่อประเมินประสิทธิภาพและหาค่าพารามิเตอร์ที่เหมาะสมของโมดูล LoRa ในการส่งข้อมูลภาพ เพื่อทดสอบและวิเคราะห์ผลกระทบของค่า Spreading Factor (SF9), Bandwidth (125 kHz), Coding Rate (4/5) และระยะเวลาหน่วง Airtime ที่มีต่อความเร็ว อัตราการสูญหายของแพ็กเกจ (Packet Loss) และความเสถียรในการส่งไฟล์รูปภาพ

1.3 ขอบเขตของโครงงาน

เพื่อให้การดำเนินโครงงานเป็นไปตามวัตถุประสงค์ที่กำหนดไว้ จึงได้กำหนดขอบเขตของการดำเนินงานออกเป็น 4 ด้านหลัก ดังนี้:

ขอบเขตด้านฮาร์ดแวร์ (Hardware Scope)

1. ไมโครคอนโทรลเลอร์: ใช้บอร์ด ESP32 DevKit จำนวน 2 บอร์ด ทำหน้าที่เป็นตัวกลางสื่อสาร (Serial Passthrough Bridge) เชื่อมต่อระหว่างคอมพิวเตอร์และโมดูล LoRa
2. โมดูลสื่อสารไร้สาย: ใช้โมดูล REYAX RYLR998 ความถี่ 868/923 MHz จำนวน 2 โมดูล (ฝั่งส่ง 1 ชุด และฝั่งรับ 1 ชุด)
3. อุปกรณ์ประมวลผล: ใช้คอมพิวเตอร์ 2 เครื่อง (Computer A เป็นฝั่งส่ง และ Computer B เป็นฝั่งรับ) เชื่อมต่อกับ ESP32 ผ่านพอร์ต USB Serial
4. การต่อวงจร: เชื่อมต่อสื่อสารระหว่าง ESP32 และ RYLR998 ผ่านพอร์ต Hardware Serial 2 (UART2) โดยใช้พิน GPIO 16 (RX2) และ GPIO 17 (TX2)

ขอบเขตด้านซอฟต์แวร์และโปรโตคอล (Software & Protocol Scope)

1. ภาษาและสภาพแวดล้อมที่ใช้:
    - ใช้ภาษา C++ (Arduino IDE) สำหรับการเขียนเฟิร์มแวร์ลงใน ESP32 เพื่อทำหน้าที่รับ-ส่งข้อมูลพาสทรู (Passthrough) แบบ Byte-by-Byte
    - ใช้ภาษา Python (Python 3.8+) ร่วมกับไลบรารี pyserial, binascii (หรือ base64) และ PIL บนคอมพิวเตอร์ทั้งสองฝั่งในการประมวลผลไฟล์ภาพ
2. การจัดการไฟล์รูปภาพ:
    - ฝั่งส่งรองรับการย่อขนาดภาพ (Resizing) และบีบอัดคุณภาพไฟล์ JPEG
    - แปลงข้อมูลไฟล์ไบนารีเป็นข้อความตัวอักษร (Hex String หรือ Base64) เพื่อส่งผ่านคำสั่ง AT Command (AT+SEND) ของโมดูล LoRa
    - ทำการซอยย่อยไฟล์ภาพออกเป็นชิ้นส่วนขนาดเล็ก (Data Chunks)
3. โปรโตคอลตรวจสอบความถูกต้อง (ARQ Protocol):
    - ระบบการสื่อสารเป็นแบบสองทาง (Two-way Communication)
    - ฝั่งรับตรวจสอบลำดับชิ้นส่วนข้อมูล หากพบชิ้นส่วนตกหล่นจะแจ้งฝั่งส่งด้วยข้อความ MISSING เพื่อให้ฝั่งส่งยิงซ่อมเฉพาะชิ้นส่วนที่ขาดหายไป (Retransmission) จนกว่าจะครบถ้วน (OK)

ขอบเขตด้านพารามิเตอร์การสื่อสารไร้สาย (LoRa Communication Scope)

1. ย่านความถี่คลื่นวิทยุ: กำหนดไว้ที่ 923 MHz (AS923) ตามข้อกำหนดการใช้งานความถี่วิทยุในประเทศไทย
2. การตั้งค่าพารามิเตอร์ (AT+PARAMETER=9,7,1,12):
    - Spreading Factor (SF): กำหนดไว้ที่ SF9 เพื่อสร้างความสมดุลระหว่างความเร็วและระยะทางการส่งข้อมูล
    - Bandwidth (BW): กำหนดไว้ที่ 125 kHz (ค่า Index 7)
    - Coding Rate (CR): กำหนดไว้ที่ 4/5 (ค่า Index 1)
    - Preamble Length: กำหนดไว้ที่ 12 Symbols
3. การระบุตัวตนและเครือข่าย: กำหนด Network ID = 18 และตั้งค่าหมายเลขอุปกรณ์ (AT+ADDRESS) เป็น 1 สำหรับฝั่งส่ง และ 2 สำหรับฝั่งรับ

ขอบเขตการทดสอบและข้อจำกัด (Testing & Limitation Scope)

1. ขนาดไฟล์รูปภาพ: รองรับการส่งไฟล์รูปภาพต้นฉบับที่มีขนาดไฟล์หลังจากย่อ/บีบอัดแล้วไม่เกิน 5 KB ถึง 10 KB เพื่อให้อยู่ในขอบเขตเวลาและ Airtime ที่เหมาะสมของคลื่นความถี่ LoRa
2. สภาพแวดล้อมการทดสอบ: ทำการทดสอบรับ-ส่งข้อมูลในระยะทางสายตา (Line-of-Sight) และภายในอาคาร เพื่อประเมินอัตราความสำเร็จในการส่งข้อมูล (Packet Loss Rate) และระยะเวลาในการส่งไฟล์ภาพ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ได้ระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกลต้นแบบ: ได้รับระบบ LoRa Serial Bridge ที่สามารถรับ-ส่งไฟล์รูปภาพข้ามคอมพิวเตอร์ผ่านคลื่นวิทยุความถี่ 923 MHz ได้จริง โดยไม่มีความเสียหายของไฟล์ข้อมูล (100% Data Integrity)
2. ได้โปรโตคอลสื่อสารสื่อสารสองทางและระบบส่งซ่อมข้อมูล (ARQ): ได้ต้นแบบอัลกอริทึมการตรวจสอบแพ็กเกจตกหล่นและการขอส่งซ่อมเฉพาะส่วนที่ขาดหาย (Automatic Repeat Request) ซึ่งสามารถนำไปประยุกต์ใช้กับการรับส่งข้อมูลไฟล์ประเภทอื่น หรือข้อมูลเซ็นเซอร์สำคัญที่ไม่ต้องการให้เกิด Data Loss
3. เข้าใจผลกระทบของพารามิเตอร์ LoRa ต่อการสื่อสารข้อมูล: ได้รับความรู้และข้อมูลเชิงทดลองเกี่ยวกับความสัมพันธ์ระหว่างค่า Spreading Factor (SF9), Bandwidth, Airtime Delay และขนาด Chunk ข้อมูล ซึ่งสามารถนำไปใช้เป็นดรรชนีอ้างอิงในการออกแบบระบบสื่อสาร LoRa ในโครงงานอื่นๆ ต่อไป
4. สามารถนำไปประยุกต์ใช้กับงานเฝ้าระวังในพื้นที่ห่างไกล (Remote Monitoring): องค์ความรู้และระบบต้นแบบนี้สามารถนำไปพัฒนาต่อยอดใช้ในงาน IoT เช่น ระบบถ่ายภาพและส่งภาพเฝ้าระวังพื้นที่ป่าไม้ แปลงเกษตรกรรม หรือพื้นที่ประสบภัยธรรมชาติที่ไม่สัญญาณอินเทอร์เน็ตหรือเซลลูลาร์ได้อย่างมีประสิทธิภาพ

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในการศึกษาวิจัยและพัฒนาระบบรับส่งรูปภาพไร้สายระยะไกลด้วย LoRa คณะผู้จัดทำได้ทำการรวบรวม ศึกษาทฤษฎี หลักการ และงานวิจัยที่เกี่ยวข้องเพื่อใช้เป็นแนวทางและพื้นฐานในการดำเนินโครงงาน โดยแบ่งเนื้อหาออกเป็นหัวข้อต่างๆ ดังนี้

2.1 เทคโนโลยีการสื่อสารไร้สาย LoRa และโปรโตคอล LoRaWAN

2.1.1 เทคโนโลยี LoRa

LoRa คือ เทคโนโลยีการสื่อสารไร้สายในระดับชั้นกายภาพ (Physical Layer) ที่พัฒนาขึ้นสำหรับการส่งข้อมูลระยะไกลและใช้พลังงานต่ำ (Low Power, Long Range) ซึ่งอาศัยเทคนิคการผสมสัญญาณคลื่นวิทยุแบบ Chirp Spread Spectrum (CSS) เทคนิคนี้ช่วยให้สัญญาณวิทยุมีความทนทานต่อสัญญาณรบกวน (Noise) และปรากฏการณ์การสะท้อนของคลื่น (Multipath Fading) สูง ทำให้สามารถส่งสัญญาณทะลุสิ่งกีดขวางในสิ่งแวดล้อมต่างๆ เช่น ตัวอาคาร หรือพื้นที่ป่าไม้ ได้ดีเยี่ยม
คุณลักษณะสำคัญของเทคโนโลยี LoRa ที่เกี่ยวข้องกับโครงงาน มีดังนี้:

- ย่านความถี่ใช้งาน (Frequency Band): LoRa ทำงานอยู่บนย่านความถี่วิทยุที่ไม่ต้องขอใบอนุญาต (Unlicensed ISM Band) ซึ่งถูกกำหนดแตกต่างกันตามกฎหมายของแต่ละภูมิภาค เช่น 868 MHz ในยุโรป, 915 MHz ในอเมริกาเหนือ และ 923 MHz (AS923) สำหรับประเทศไทยและประเทศในแถบเอเชีย
- การส่งข้อมูลระยะไกล (Long Range): สามารถส่งข้อมูลได้ระยะทางตั้งแต่ 2–5 กิโลเมตรในเขตเมืองที่มีสิ่งกีดขวางแน่นหนา และได้ไกลถึง 10–15 กิโลเมตรในพื้นที่โล่งแจ้ง (Line-of-Sight)
- การใช้อัตราส่งข้อมูลต่ำ (Low Data Rate): LoRa ถูกออกแบบมาเพื่อรับ-ส่งแพ็กเกจข้อมูลขนาดเล็ก จึงมีอัตราความเร็วการส่งข้อมูล (Bit Rate) ที่ค่อนข้างต่ำ ตั้งแต่ไม่กี่ร้อย bps ไปจนถึงประมาณ 21.8 kbps ขึ้นอยู่กับการตั้งค่าพารามิเตอร์

2.1.2 ค่าพารามิเตอร์ทางกายภาพของ LoRa

การปรับแต่งพารามิเตอร์ในการส่งคลื่นวิทยุ LoRa จะส่งผลต่อระยะทาง ความเร็ว และระยะเวลาในการแผ่คลื่นสัญญาณกลางอากาศ (Time-on-Air) โดยตรง ซึ่งพารามิเตอร์หลักประกอบด้วย:

1. Spreading Factor (SF):

คือ ระดับการขยายสัญญาณวิทยุ มีค่าตั้งแต่นี้ SF7 ถึง SF12

- SF ค่าต่ำ (เช่น SF7): ให้ความเร็วในการส่งข้อมูลสูง อัตรา Time-on-Air สั้น แต่ระยะทางการส่งจะใกล้ขึ้น
- SF ค่าสูง (เช่น SF12): ให้ระยะทางการส่งข้อมูลที่ไกลมากและทะลุทะลวงได้ดี แต่ความเร็วจะช้าลงอย่างมาก
- *ในโครงงานนี้เลือกใช้ SF9 เพื่อสร้างความสมดุลระหว่างระยะทางและความเร็วที่จำเป็นต่อการส่งข้อมูลภาพ*

2. Bandwidth (BW):

คือ ความกว้างของช่องสัญญาณวิทยุ ค่ามาตรฐานที่นิยมใช้คือ 125 kHz, 250 kHz และ 500 kHz การขยาย Bandwidth กว้างขึ้นจะช่วยเพิ่มความเร็วในการส่งข้อมูล แต่จะลดความไวในการรับสัญญาณ (Receiver Sensitivity) ลงเล็กน้อย

3. Coding Rate (CR):

คือ อัตราส่วนการแทรกรหัสแก้ไขข้อผิดพลาด (Forward Error Correction - FEC) มีค่าตั้งแต่ 4/5 (CR1) ถึง 4/8 (CR4) เพื่อช่วยให้ภาครับสามารถแก้ไขข้อมูลที่ผิดพลาดกลางทางได้โดยไม่ต้องส่งใหม่

4. Preamble Length:

คือ สัญญาณนำทางที่ใส่ไว้ส่วนหัวของแพ็กเกจวิทยุ เพื่อสะกิดให้ภาครับตื่นขึ้นเตรียมพร้อมรับข้อมูลจริงที่กำลังจะตามมา

2.1.3 ความแตกต่างระหว่าง LoRa และ LoRaWAN

แม้คำว่า "LoRa" และ "LoRaWAN" มักถูกเรียกคู่กัน แต่ในทางสถาปัตยกรรมระบบทั้งสองมีความแตกต่างกันอย่างชัดเจน:

ประเด็นเปรียบเทียบ	LoRa (Physical Layer)	LoRaWAN (MAC & Network Layer)
ระดับชั้นระบบ (Layer)	Physical Layer: กำหนดลักษณะคลื่นวิทยุ การผสมสัญญาณ (CSS) และฮาร์ดแวร์	MAC Layer & Network Layer: กำหนดโครงสร้างเครือข่าย โปรโตคอล และระบบรักษาความปลอดภัย
สถาปัตยกรรม (Architecture)	รองรับการเชื่อมต่อแบบ Point-to-Point (P2P) เช่น อุปกรณ์ต่ออุปกรณ์โดยตรง	ต้องสื่อสารผ่าน Star Topology (End Device → Gateway → Network Server)
การควบคุมคำสั่ง	นิยมส่งผ่านโมดูลที่ควบคุมด้วยคำสั่ง AT Command (เช่น REYAX RYLR998)	ต้องส่งผ่านเกตเวย์และบริหารจัดการสิทธิ์ผู้ใช้งานบนระบบ Cloud / Network Server
ความเหมาะสมในโครงงาน	ตรงกับโครงงานนี้: เนื่องจากเป็นการรับ-ส่งไฟล์ระหว่างสองอุปกรณ์โดยตรง ไม่ต้องพึ่งพาเกตเวย์	เหมาะกับระบบเซ็นเซอร์ขนาดใหญ่ที่มีอุปกรณ์รับ-ส่งหลายร้อยตัว

2.1.3 ความแตกต่างระหว่าง LoRa และ LoRaWAN

2.2 คุณลักษณะทางเทคนิคของโมดูล REYAX RYLR998

โมดูลสื่อสารไร้สาย REYAX RYLR998 เป็นโมดูลรับ-ส่งสัญญาณ LoRa ที่ใช้ชิปประมวลผลหลัก Semtech SX1262 ร่วมกับตัวประมวลผล STM32 ภายในตัว ถูกออกแบบมาให้ใช้งานง่ายด้วยการควบคุมผ่านชุดคำสั่งมาตรฐาน AT Command ผ่านอินเทอร์เฟซการสื่อสารอนุกรม (UART) ทำให้เหมาะสำหรับการนำมาทำเป็นสายสัญญาณเสมือน (Serial Bridge) ในการรับส่งข้อมูลระยะไกล

2.2.1 คุณสมบัติทางไฟฟ้าและประสิทธิภาพคลื่นวิทยุ

1. ย่านความถี่ใช้งาน (Frequency Range):

รองรับช่วงความถี่ 868 MHz ถึง 915 MHz และสามารถตั้งค่าการทำงานย่านความถี่ 923 MHz (AS923) สำหรับประเทศไทยได้ผ่านคำสั่ง AT+BAND=923000000

2. แรงดันไฟฟ้ากระแสตรง (Operating Voltage): ทำงานที่ระดับแรงดันไฟเลี้ยง 3.0V ถึง 3.6V DC (ค่าปกติ 3.3V) ทำให้ต่อใช้งานกับพินจ่ายไฟ 3.3V ของไมโครคอนโทรลเลอร์ ESP32 ได้โดยตรง
3. กำลังส่งออกคลื่นวิทยุ (TX Transmit Power):

ตั้งค่ากำลังส่งได้สูงสุดถึง +22 dBm (ประมาณ 160 mW) ช่วยให้ส่งสัญญาณได้ระยะไกลและทะลุทะลวงสิ่งกีดขวางได้สูง

4. ความไวในการรับสัญญาณ (RX Sensitivity):

มีความไวในการรับสัญญาณสูงถึง -129 dBm ถึง -148 dBm (ขึ้นอยู่กับค่า Spreading Factor ที่ตั้งไว้) ช่วยให้รับแพ็กเกจข้อมูลขนาดเล็กที่มีสัญญาณเบาได้อย่างแม่นยำ

5. อินเทอร์เฟซการเชื่อมต่อ (UART Interface): สื่อสารผ่านโปรโตคอล Serial (UART) ระดับลอจิก 3.3V TTL ที่อัตราความเร็วเริ่มต้น (Default Baud Rate) 115200 bps (Data bits: 8, Stop bit: 1, Parity: None)

2.2.2 รูปแบบโครงสร้างชุดคำสั่ง AT Command ที่สำคัญ

โมดูล RYLR998 ใช้การส่งข้อความผ่าน Serial ด้วยคำสั่ง AT Command โดยต้องลงท้ายสายอักษรด้วยค่า \r\n (CRLF) เสมอ คำสั่งหลักที่นำมาใช้ในโครงงานประกอบด้วย:

คำสั่ง AT Command	หน้าที่การทำงาน	ตัวอย่างการใช้งานในโครงงาน
AT+BAND	กำหนดความถี่การสื่อสาร (หน่วย Hz)	AT+BAND=923000000 (ตั้งค่าความถี่ 923 MHz)
AT+ADDRESS	กำหนดหมายเลขไอดีประจวบจำตัวโมดูล (0–65535)	AT+ADDRESS=1 (ฝั่งส่ง) / AT+ADDRESS=2 (ฝั่งรับ)
AT+NETWORKID	กำหนดวงเครือข่ายวิทยุ (0–18)	AT+NETWORKID=18 (ตั้งวงเครือข่ายเดียวกัน)
AT+PARAMETER	กำหนดค่า SF, BW, CR และ Preamble	AT+PARAMETER=9,7,1,12 (ตั้งค่า SF9, BW 125kHz)
AT+SEND	ส่งข้อมูลไปยังอุปกรณ์เป้าหมาย	AT+SEND=<Addr>,<Length>,<Data>

2.2.2 รูปแบบโครงสร้างชุดคำสั่ง AT Command ที่สำคัญ

2.2.3 รูปแบบโครงสร้างแพ็กเกจการรับข้อมูล (+RCV)

เมื่อโมดูล RYLR998 ภาครับสามารถดักจับคลื่นวิทยุและถอดรหัสแพ็กเกจข้อมูลได้สำเร็จ ตัวโมดูลจะส่งข้อความพ่นออกมาทางพอร์ต UART ในรูปแบบมาตรฐานดังนี้:

$$+\text{RCV} = \langle \text{Address} \rangle, \langle \text{Data_Length} \rangle, \langle \text{Data} \rangle, \langle \text{RSSI} \rangle, \langle \text{SNR} \rangle$$

- Address: หมายเลขไอดีประจำตัวของฝั่งส่งที่ยิงแพ็กเกจนี้มา
- Data_Length: ความยาวของชุดข้อมูลตัวอักษรที่ได้รับ

Data: เนื้อหาข้อมูลจริงที่ถูกส่งมา เช่น START:2450, 0001:414243 หรือคำสั่ง CHECK / MISSING

- RSSI (Received Signal Strength Indicator): ค่าความแรงของสัญญาณวิทยุที่รับได้ มีหน่วยเป็น dBm (ยิ่งเข้าใกล้ 0 ยิ่งมีสัญญาณแรง เช่น -60 dBm ดีกว่า -110 dBm)
- SNR (Signal-to-Noise Ratio): อัตราส่วนระหว่างสัญญาณจริงต่อสัญญาณรบกวน มีหน่วยเป็น dB (ยิ่งเป็นค่าบวกมาก แสดงว่าสัญญาณคมชัดไม่มีสัญญาณรบกวน)

2.3 ไมโครคอนโทรลเลอร์ ESP32 และการสื่อสารอนุกรม

2.3.1 คุณลักษณะของไมโครคอนโทรลเลอร์ ESP32

ESP32 เป็นบอร์ดไมโครคอนโทรลเลอร์ประสิทธิภาพสูงที่พัฒนาโดยบริษัท Espressif Systems มีหน่วยประมวลผลหลักเป็น Tensilica Xtensa LX6 แบบ Dual-Core ความเร็วนาฬิกาสูงสุด 240 MHz มีหน่วยความจำ SRAM ภายในขนาด 520 KB และ Flash Memory ขนาด 4 MB ขึ้นไป
ในการดำเนินโครงงานนี้ ESP32 ถูกเลือกนำมาใช้งานในลักษณะ Bridge Layer ทำหน้าที่เป็นตัวกลาง (Serial Passthrough Bridge) เชื่อมต่อระหว่างคอมพิวเตอร์ผ่านพอร์ต USB Serial กับโมดูลสื่อสาร LoRa RYLR998 ผ่านพอร์ต Hardware Serial เนื่องจาก ESP32 มีจุดเด่นด้านความเร็วในการรับ-ส่งข้อมูลอนุกรมพร้อมกันหลายพอร์ต และมีราคาประหยัด

2.3.2 การสื่อสารอนุกรมแบบอนุกรมระดับฮาร์ดแวร์

การสื่อสารข้อมูลอนุกรมแบบไม่ประสานเวลา หรือ UART (Universal Asynchronous Receiver-Transmitter) เป็นโปรโตคอลการสื่อสารแบบสองทางที่ใช้สายสัญญาณหลักเพียง 2 เส้น คือ สายส่งข้อมูล (TX - Transmit) และสายรับข้อมูล (RX - Receive) สื่อสารผ่านระดับแรงดันลอจิก (Logic Level 3.3V TTL) ชิปประมวลผล ESP32 มีวงจรฮาร์ดแวร์ UART ภายใน (Hardware Serial) ทั้งหมด 3 ช่อง ได้แก่:

1. UART0: (ใช้พิน GPIO 1 และ GPIO 3) สงวนไว้สำหรับการอัปโหลดโปรแกรมจากคอมพิวเตอร์ผ่านพอร์ต USB-to-Serial และการแสดงผลบน Serial Monitor
2. UART1: มักถูกนำไปใช้งานภายในหรือกำหนดพินเชื่อมต่อใหม่ตามต้องการ
3. UART2: (ใช้พิน GPIO 16 สำหรับ RX2 และ GPIO 17 สำหรับ TX2) ถูกเลือกนำมาใช้ในโครงงานนี้เพื่อทำการสื่อสารกับโมดูล REYAX RYLR998 โดยเฉพาะ

2.3.3 หลักการทำงานแบบบริดจ์ส่งผ่านข้อมูล

ในโครงงานนี้ เฟิร์มแวร์ Arduino C++ บน ESP32 ไม่ได้ทำการคำนวณโลจิกของภาพหรือการจัดลำดับ ARQ แต่ถูกออกแบบให้ทำหน้าที่เป็น "สะพานส่งผ่านข้อมูลเสมือน" (Transparent Passthrough Bridge) แบบ Byte-by-Byte ระหว่างพอร์ต Serial (USB) และ Serial2 (UART2)
การทำงานแบ่งออกเป็น 2 ทิศทางขนานกัน:

1. ทิศทางขาออก (USB → LoRa): อ่านข้อมูลข้อความ/คำสั่ง AT Command ที่ส่งมาจากสคริปต์ Python บนคอมพิวเตอร์ผ่าน USB Serial แล้วเขียนส่งต่อไปยังพอร์ต Serial2 ออกสู่โมดูล RYLR998 ทันที
2. ทิศทางขาเข้า (LoRa → USB): อ่านข้อมูลแพ็กเกจสัญญาณวิทยุ +RCV=... ที่รับมาจากโมดูล RYLR998 ผ่านพอร์ต Serial2 แล้วเขียนส่งต่อขึ้นไปยังสคริปต์ Python บนคอมพิวเตอร์ผ่าน USB Serial

```
// ตัวอย่างโลจิก Passthrough บน ESP32
void loop() {
  if (Serial.available()) {
    Serial2.write(Serial.read()); // คอมพิวเตอร์ -> LoRa Module
  }
  if (Serial2.available()) {
    Serial.write(Serial2.read()); // LoRa Module -> คอมพิวเตอร์
  }
}
```

2.3.4 การตั้งค่าอัตราเร็วสื่อสาร (Baud Rate) และการขยายบัฟเฟอร์ข้อมูล

Baud Rate (115200 bps): ระบบได้รับการกำหนดให้อัตราความเร็วสื่อสารข้อมูลอนุกรมอยู่ที่ 115200 bits per second (bps) ทั้งฝั่งพอร์ต USB และพอร์ต Hardware Serial 2 เพื่อให้ส่งผ่านข้อมูลข้อความ Hex/Base64 ได้รวดเร็วโดยไม่เกิดปัญหาคอขวด (Bottleneck)
การจัดการ RX Buffer (Serial RX Buffer Expansion):** เนื่องจากชุดข้อมูลแพ็กเกจหรือคำสั่งตอบกลับการส่งซ่อม (เช่น คำสั่งแจ้งชิ้นส่วนขาดหาย `MISSING:1,2,5,...`) อาจมีความยาวเกินกว่าขนาดบัฟเฟอร์มาตรฐานเดิมของไมโครคอนโทรลเลอร์ จึงสามารถปรับเพิ่มขนาดพื้นที่สำรองข้อมูลในหน่วยความจำ RAM ของ ESP32 ผ่านคำสั่ง `Serial2.setRxBufferSize(1024)` เพื่อป้องกันปัญหาข้อมูลสูญหายจากภาวะบัฟเฟอร์ล้น (Buffer Overflow) ได้

2.4 การบีบอัดไฟล์ภาพ JPEG และการแปลงรหัส

เนื่องจากอัตราการส่งข้อมูลผ่านคลื่นวิทยุ LoRa มีความเร็วค่อนข้างต่ำ (Low Bitrate) และมีข้อจำกัดด้านระยะเวลาการส่งสัญญาณกลางอากาศ (Time-on-Air / Duty Cycle) การส่งข้อมูลไฟล์ภาพดิบ (Raw Binary Image) โดยตรงจึงไม่สามารถทำได้ทางปฏิบัติ การเตรียมไฟล์ภาพและปรับเปลี่ยนรูปแบบข้อมูลก่อนส่งจึงเป็นขั้นตอนที่สำคัญที่สุดของการประยุกต์ใช้งานระบบ

2.4.1 การบีบอัดและปรับขนาดภาพ JPEG

สคริปต์ Python ฝั่งส่ง (send_image.py) ประมวลผลไฟล์ภาพดิบด้วยไลบรารี PIL (Pillow) ผ่านกระบวนการเตรียมไฟล์ 2 ขั้นตอนหลัก:

1. การลดขนาดพิกเซล (Resizing): ใช้คำสั่ง img.thumbnail((350, 350)) เพื่อปรับย่อขนาดความกว้างและความสูงของภาพให้อยู่ในขอบเขตไม่เกิน 350×350 พิกเซล โดยยังคงรักษาอัตราส่วนภาพเดิม (Aspect Ratio) ไว้ เพื่อลดจำนวนพิกเซลทั้งหมดของภาพลงก่อนการบันทึก
2. การบีบอัดคุณภาพไฟล์ JPEG (JPEG Compression): ใช้คำสั่ง img.save(..., quality=45) เพื่อปรับลดค่าความคมชัดของภาพลงเหลือ 45% ซึ่งเป็นระดับที่ช่วยลดขนาดไฟล์ข้อมูลลงได้อย่างมีประสิทธิภาพสูง โดยที่รายละเอียดสำคัญของภาพยังคงมองเห็นได้อย่างชัดเจน ผลลัพธ์จากกระบวนการนี้ช่วยให้ขนาดไฟล์ภาพ test.jpg ลดลงมาเหลือเพียงประมาณ 1 KB ถึง 5 KB เท่านั้น

2.4.2 การย่อยชิ้นส่วนข้อมูล

หลังจากบีบอัดไฟล์ภาพจนได้ข้อมูลไบนารี (bytes) แล้ว สคริปต์ Python จะทำการแบ่งข้อมูลภาพออกเป็นชิ้นส่วนขนาดเล็ก (Data Chunks) เพื่อใส่ในแพ็กเกจส่งของ LoRa:

- การคำนวณจำนวนชิ้นส่วน (Total Chunks): กำหนดขนาดชิ้นส่วนสูงสุดต่อแพ็กเกจ เช่น CHUNK_SIZE = 100 ไบต์ แล้วคำนวณจำนวนแพ็กเกจทั้งหมดผ่านสูตร:

$$\text{Total Chunks} = \left\lfloor \frac{\text{Total Bytes} + \text{CHUNK_SIZE} - 1}{\text{CHUNK_SIZE}} \right\rfloor$$

- การระบุลำดับแพ็กเกจ (Sequence Numbering): ชิ้นส่วนข้อมูลทุกชิ้นจะถูกกำกับด้วยเลขลำดับ 4 หลัก เช่น 0000:, 0001:, 0002: นำหน้าชุดข้อมูลเสมอ เพื่อให้ฝั่งรับสามารถนำข้อมูลไปจัดเรียงเข้าตำแหน่งเดิมได้อย่างถูกต้อง แม้แพ็กเกจจะสลับลำดับหรือต้องมีการส่งซ่อมเฉพาะบางแพ็กเกจก็ตาม

2.4.3 รูปแบบการแปลงรหัสข้อมูล

เนื่องจากชุดคำสั่ง AT Command ของโมดูล REYAX RYLR998 รับข้อมูลในรูปแบบตัวอักษรข้อความ (Text-based String) และไม่รองรับการส่งค่า Raw Binary โดยตรง (เช่น ตัวอักษรควบคุม 0x00 หรือ End-of-Line \r\n ซึ่งอาจทำให้โมดูลตีความคำสั่งผิดพลาด) ข้อมูลไบนารีของภาพจึงต้องถูกแปลงรหัสเป็นข้อความก่อนส่งเสมอ:

1. การแปลงรหัสแบบฐานสิบหก (Hexadecimal / Hex Encoding)

- หลักการ: แปลงข้อมูล 1 ไบต์ไบนารี (8 bits) ให้กลายเป็นตัวอักษร Hex จำนวน 2 ตัวอักษร (เช่น ค่าไบนารี 0xFF แปลงเป็นข้อความ "FF") ผ่านฟังก์ชัน binascii.hexlify()
- ข้อดี: ปลอดภัยต่อการส่งผ่านคำสั่ง AT Command 100% อ่านจับแก้ง่าย
- ข้อเสีย: เกิดภาระข้อมูลเพิ่มขึ้น (Overhead) เท่าตัว หรือ 100% (ไฟล์ภาพขนาด 3 KB จะกลายเป็นข้อความยาว 6 KB)

2. การแปลงรหัสแบบ Base64 (Base64 Encoding)

- หลักการ: แปลงข้อมูลกลุ่มละ 3 ไบต์ไบนารี (24 bits) ให้กลายเป็นตัวอักษรชุด ASCII จำนวน 4 ตัวอักษร ผ่านไลบรารี base64
- ข้อดี: ช่วยลดภาระข้อมูลลง โดยมี Overhead เพียง 33% เท่านั้น (ไฟล์ภาพขนาด 3 KB จะกลายเป็นข้อความยาวประมาณ 4 KB) ช่วยลดเวลาส่งผ่านคลื่นวิทยุ (Time-on-Air) ลงได้เกือบครึ่งหนึ่งเมื่อเทียบกับ Hex Encoding

เมื่อฝั่งรับ (receive_image.py) ได้รับข้อความชิ้นส่วนข้อมูลครบถ้วน จะทำการถอดรหัสข้อความกลับไปเป็นข้อมูลไบนารีดิบด้วย binascii.unhexlify() (หรือ base64.b64decode()) แล้วทำการเขียนบันทึกไฟล์ลงดิสก์กลับคืนเป็นไฟล์ภาพ received_image.jpg ที่สมบูรณ์

2.5 เทคนิคการแบ่งแพ็กเก็ตข้อมูล

เนื่องจากการสื่อสารไร้สายด้วยคลื่นวิทยุ LoRa ผ่านโมดูล REYAX RYLR998 มีข้อจำกัดเรื่องขนาดของบัฟเฟอร์คำสั่ง AT+SEND (ส่งข้อมูลได้สูงสุดไม่เกิน 240 ตัวอักษรต่อครั้ง) และข้อจำกัดด้านเวลาแผ่คลื่นสัญญาณกลางอากาศ (Time-on-Air) เพื่อไม่ให้กระทบต่อข้อกำหนด Duty Cycle ของย่านความถี่วิทยุ การส่งไฟล์รูปภาพที่มีขนาดหลายกิโลไบต์ (KB) จึงต้องอาศัย เทคนิคการย่อยแบ่งแพ็กเก็ตข้อมูล (Chunking Protocol) ในระดับประยุกต์ (Application Layer)

2.5.1 โครงสร้างแพ็กเก็ตข้อมูลและส่วนหัว

เพื่อให้ฝั่งรับสามารถแยกแยะความแตกต่างระหว่างข้อความควบคุมระบบและข้อมูลรูปภาพได้ ทุกแพ็กเกจที่ส่งจากฝั่งส่ง (send_image.py) ผ่านคำสั่ง AT+SEND จะถูกกำหนดโครงสร้างอย่างเป็นระบบดังนี้:

[โครงสร้างแพ็กเก็ตข้อมูลรูปภาพ] Sequence Number (4-digit) + ":" + Encoded Data Payload (ตัวอย่าง: "0001:41424344...") ``` Sequence Number (ส่วนหัวแพ็กเก็ต): เป็นตัวเลขลำดับจำนวน 4 หลัก (เช่น `0000`, `0001`, `0002` ถึง `9999`) ทำหน้าที่ระบุตำแหน่งของชิ้นส่วนข้อมูลในไฟล์ภาพ[cite: 2] ช่วยให้ฝั่งรับนำข้อมูลไปประกอบลงตำแหน่งใน Dictionary หรือ Array ได้อย่างถูกต้องแม้แพ็กเกจจะส่งมาสลับลำดับ Delimiter (`:`): เครื่องหมายสัญลักษณ์แบ่งแยก (Separator) เพื่อให้ฝั่งรับทำการแยกแยะ (Split) เลขลำดับออกจากเนื้อหาข้อมูลจริงได้อย่างรวดเร็วEncoded Data Payload: เนื้อหาข้อมูลรูปภาพที่ผ่านการแปลงรหัสเป็น Hex String หรือ Base64 มีขนาดคงที่ตามที่กำหนดไว้ (เช่น 100 ไบต์)

2.5.2 ขั้นตอนการซอยย่อยข้อมูล

กระบวนการย่อยข้อมูลฝั่งส่งมีลำดับการประมวลผลตามขั้นตอนดังนี้

1. การคำนวณจำนวนชิ้นส่วนทั้งหมด (Total Chunks Calculation)

คำนวณจำนวนชิ้นส่วนที่ต้องส่งทั้งหมดจากขนาดของข้อมูลข้อความภาพที่แปลงรหัสแล้ว สอดคล้องตามสูตร

$$\text{Total Chunks} = \left\lceil \frac{\text{Total Encoded Bytes}}{\text{CHUNK_SIZE}} \right\rceil$$

2. การสไลซ์ข้อมูล (String Slicing)

ใช้เทคนิคการตัดแบ่งสายอักษร (String Slicing) ใน Python ตัดข้อมูลข้อความออกมาทีละช่วงความยาวเท่ากับ `CHUNK_SIZE` (เช่น ทีละ 100 ถึง 150 ตัวอักษร)

3. การประกอบเป็นแพ็กเก็ตสมบูรณ์:

รวมเลขลำดับ 4 หลัก เครื่องหมาย และข้อความย่อยเข้าด้วยกัน แล้วนำไปใส่ในคำสั่ง `AT+SEND=<Address>,<Length>,<Packet>` เพื่อส่งออกทางพอร์ต Serial ไปยังโมดูล

2.5.3 การจัดการเวลาหน่วงแอร์ไทม์

การส่งแพ็กเก็ตข้อมูลติดต่อกันถี่เกินไปอาจทำให้เกิดปัญหาบัฟเฟอร์ล้นในตัวโมดูล LoRa หรือคลื่นวิทยุชนกัน (Collision) กลางอากาศในเทคนิค Chunking Protocol จึงจำเป็นต้องมีการแทรกระยะเวลาหน่วงระหว่างแพ็กเก็ต (Inter-packet Delay / Airtime Sleep)
การตั้งค่าเวลารอคอย (`time.sleep`) สคริปต์ฝั่งส่งจะทำการหน่วงเวลารอระหว่าง 0.5 ถึง 0.8 วินาที หลังจากยิงคำสั่ง `AT+SEND` แต่ละชิ้นส่วน
วัตถุประสงค์เพื่อเปิดโอกาสให้โมดูล RYLR998 ฝั่งส่งทำการส่งคลื่นวิทยุออกไปจนจบแพ็กเกจ (Time-on-Air) และให้โมดูลฝั่งรับมีเวลาเพียงพอในการถอดรหัสแพ็กเกจ พ่นข้อมูลออกทาง UART2 และให้ ESP32 นำข้อมูลส่งขึ้นคอมพิวเตอร์ผ่าน USB ได้ทันโดยไม่เกิดข้อมูลตกหล่น

2.6 งานวิจัยและเอกสารที่เกี่ยวข้อง

ในการศึกษาระบบรับ-ส่งไฟล์รูปภาพไร้สายด้วยคลื่นวิทยุ LoRa และการออกแบบโปรโตคอลการสื่อสาร ผู้จัดทำได้ทำการทบทวนเอกสารและงานวิจัยที่เกี่ยวข้อง เพื่อใช้เป็นแนวทางและหลักการในการพัฒนาระบบ ดังนี้:

2.6.1 การประยุกต์ใช้เทคโนโลยี LoRa สำหรับการรับส่งข้อมูลรูปภาพ

การส่งไฟล์รูปภาพผ่านเครือข่าย LoRa เป็นความท้าทายสำคัญเนื่องจาก LoRa มีข้อจำกัดด้านแบนด์วิดท์ (Bandwidth) และอัตราส่งข้อมูล (Data Rate) จากการศึกษาของ Sinha et al. ได้เสนอแนวทางการประยุกต์ใช้ LoRa ในงานเฝ้าระวังพื้นที่ห่างไกล โดยทำการลดความละเอียดและบีบอัดไฟล์ภาพด้วยอัลกอริทึม JPEG ก่อนแปลงเป็นข้อความรหัส ทำให้สามารถส่งภาพผ่านคลื่นวิทยุระยะไกลได้สำเร็จ สอดคล้องกับแนวทางของโครงงานนี้ที่ใช้การบีบอัดคุณภาพภาพ (Quality 45%) และปรับขนาดพิกเซลให้อยู่ในระดับไม่เกิน 350×350 พิกเซล เพื่อให้ได้ไฟล์ขนาดเล็กประมาณ 1–5 KB

2.6.2 การพัฒนากลไกโปรโตคอลตรวจสอบความถูกต้องและส่งซ่อมข้อมูล

ในการสื่อสารข้อมูลไฟล์แบบไร้สาย การสูญเสียแพ็กเกจ (Packet Loss) แม้เพียงแพ็กเกจเดียวอาจทำให้ไฟล์ภาพเสียหายจนไม่สามารถเปิดได้ งานวิจัยของ Pham (2019) เรื่อง *"Reliable Data Transfer for LoRa-based IoT Applications"* ได้นำเสนอการใช้โปรโตคอล ARQ (Automatic Repeat Request) แบบประยุกต์สำหรับอุปกรณ์ IoT ที่ใช้ LoRa โดยให้ภาครับส่งสัญญาณ ACK หรือรายการแพ็กเกจที่ตกหล่นกลับไปยังฝั่งส่ง เพื่อขอให้ยิงซ่อมเฉพาะชิ้นส่วนที่ขาดหายไป แนวคิดนี้ถูกนำมาประยุกต์ใช้ในระบบโปรโตคอลของโครงงานนี้ ผ่านการส่งคำสั่ง CHECK และการตอบกลับรายการชิ้นส่วนที่ตกหล่น เช่น MISSING:2,5,9 เพื่อให้ฝั่งส่งยิงซ่อมเฉพาะชิ้นส่วนดังกล่าว ทำให้ได้ความถูกต้องของไฟล์ปลายทาง 100%

2.6.3 การศึกษาผลกระทบพารามิเตอร์ Spreading Factor และแอร์ไทม์

จากการศึกษาเอกสารทางเทคนิคของ Semtech (SX1262) และคู่มือการใช้งานโมดูล REYAX RYLR998 พบว่าค่า Spreading Factor (SF) มีผลโดยตรงต่อระยะเวลาที่สัญญาณแผ่อยู่ในอากาศ (Time-on-Air) และความไวในการรับสัญญาณ (Sensitivity) การตั้งค่า SF10–SF12 ให้ระยะทางที่ไกลขึ้นแต่ต้องแลกด้วยเวลาการส่งที่นานขึ้นอย่างมาก ในขณะที่ SF7 ให้ความเร็วสูงแต่อาจหลุดจากคลื่นรบกวนได้ง่าย โครงงานนี้จึงเลือกอ้างอิงการตั้งค่าพารามิเตอร์ SF9 (BW 125 kHz, CR 4/5) ซึ่งเป็นจุดสมดุลที่เหมาะสมสำหรับการส่งแพ็กเกจข้อมูลรูปภาพที่ซอยย่อยแล้ว ให้สามารถส่งผ่านได้รวดเร็วและทนทานต่อสัญญาณรบกวนในสภาพแวดล้อมจริง

2.6.4 การประยุกต์ใช้ไมโครคอนโทรลเลอร์เป็น Serial Passthrough Bridge

งานวิจัยด้านระบบฝังตัว (Embedded Systems) หลายฉบับได้นำเสนอการประยุกต์ใช้ไมโครคอนโทรลเลอร์ ESP32 เป็นสะพานเชื่อมต่อข้อมูล (Serial Bridge) เนื่องจากมีพอร์ต Hardware UART หลายช่อง และมีความเร็วนาฬิกาสูง การอ้างอิงโครงสร้างแบบ Passthrough ช่วยลดภาระการประมวลผล (Processing Overhead) บนไมโครคอนโทรลเลอร์ โดยให้ ESP32 ส่งผ่านข้อมูล Byte-by-Byte ระหว่าง USB Serial และ UART2 และยกภาระการคำนวณซับซ้อน เช่น การจัดการ Dictionary ภาพและการทำ ARQ ไปไว้ที่ภาษา Python บนคอมพิวเตอร์ ทำให้ระบบทำงานได้รวดเร็วและมีความเสถียรสูง

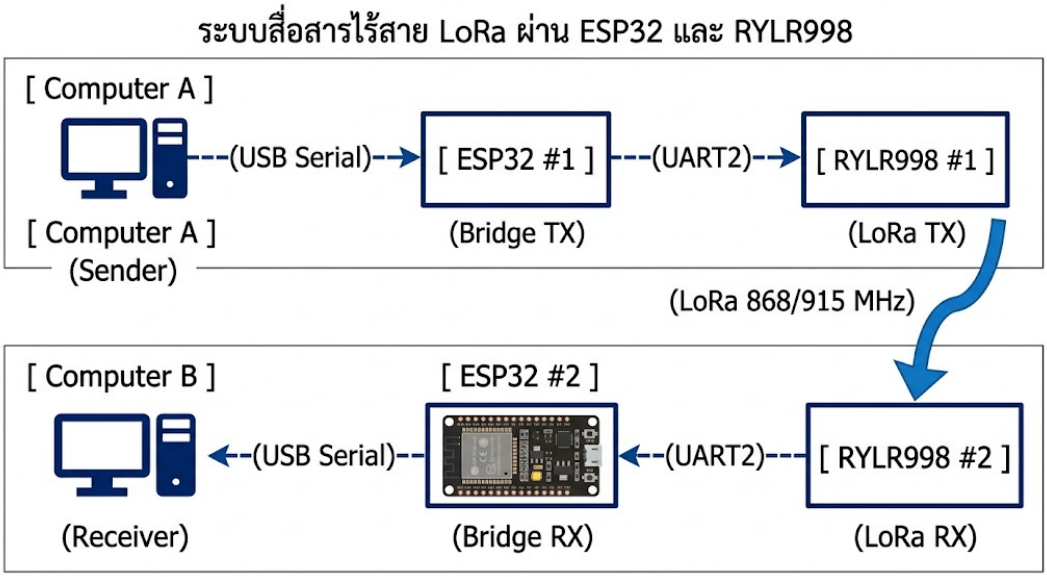
บทที่ 3

การออกแบบและพัฒนาระบบ

ในบทนี้จะกล่าวถึงรายละเอียดการออกแบบและพัฒนาระบบรับส่งรูปภาพไร้สายระยะไกลด้วย LoRa โดยแบ่งโครงสร้างการทำงานออกเป็นทั้งส่วนฮาร์ดแวร์ (Hardware Design) โปรโตคอลการสื่อสาร (Data Protocol Design) และซอฟต์แวร์ควบคุม (Software Implementation) ทั้งฝั่งคอมพิวเตอร์และไมโครคอนโทรลเลอร์ ESP32

3.1 สถาปัตยกรรมรวมของระบบ

ระบบได้รับการออกแบบให้ทำงานเป็น LoRa Serial Bridge เพื่อก้าวข้ามข้อจำกัดของโมดูล LoRa ที่รับส่งได้เฉพาะสายธารตัวอักษร (Text Stream) ขนาดเล็ก โดยแบ่งโครงสร้างการทำงานออกเป็น 2 ฝั่งหลัก คือ ฝั่งส่ง (Transmitting Station - Computer A) และฝั่งรับ (Receiving Station - Computer B)



3.1 สถาปัตยกรรมรวมของระบบ

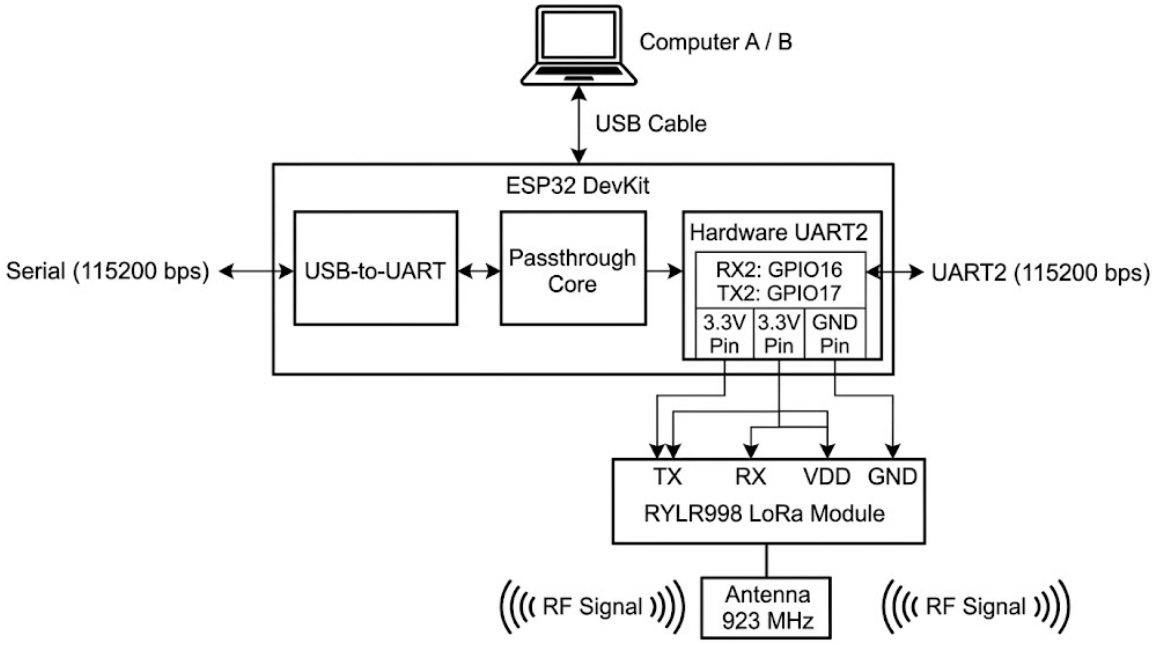
3.1.1 การแบ่งระดับชั้นการทำงาน (System Layers) ระบบแบ่งออกเป็น 3 ระดับชั้นหลักทั้งฝั่งส่ง

(Transmitter) และฝั่งรับ (Receiver) ดังนี้: 1. ชั้นการประมวลผล (Compute Layer - Python Application):ฝั่งส่ง (Computer A):ทำหน้าที่ย่อขนาดพิกเซล ย่อคุณภาพภาพ JPEG และแปลงรหัสไฟล์ไบนารีเป็นข้อความตัวอักษร (Hex/Base64) จากนั้นทำการแบ่งไฟล์ออกเป็นชิ้นส่วนขนาดเล็ก (Data Chunks) กำหนดหมายเลขลำดับ (Sequence Header) และคุมจังหวะโปรโตคอล ARQ
ฝั่งรับ (Computer B) ทำหน้าที่ดักอ่านแพ็กเกจ ดึงหมายเลขลำดับและข้อมูลพายโหลด (Payload) จัดเก็บลงใน Dictionaryตรวจสอบความถูกต้องของชิ้นส่วน (Verification) ส่งสัญญาณแจ้งเตือนขอส่งซ่อมชิ้นส่วนที่ตกหล่น (Retransmission) และแปลงรหัสข้อความกลับเป็นไฟล์รูปภาพ JPEG เพื่อบันทึกลงดิสก์ชั้นตัวกลางส่งผ่านข้อมูล (Bridge Layer - ESP32 Firmware) ไมโครคอนโทรลเลอร์ ESP32 ทั้งสองฝั่งทำหน้าที่เป็น "สะพานส่งผ่านข้อมูลเสมือน" (Transparent Passthrough Serial Bridge) ไม่มีการคำนวณหรือแก้ไขโลจิกของภาพบน ESP32 ทำหน้าที่เพียงคอยอ่านชุดข้อมูลสตรีมอนุกรมที่รับมาจากพอร์ต USB Serial แล้วส่งต่อออกไปยังพอร์ต Hardware Serial 2 (UART2) เพื่อเข้าโมดูล LoRa หรือรับข้อมูลวิทยุจากโมดูล LoRa แล้วพ่นกลับขึ้นพอร์ต USB ไปยังคอมพิวเตอร์แบบ Byte-by-Byte โดยตรง ชั้นกายภาพสื่อสารไร้สาย (Physical Layer - REYAX RYLR998 LoRa)โมดูล LoRa RYLR998 ทำหน้าที่แปลงชุดข้อความตัวอักษรคำสั่ง `AT+SEND` ที่ได้รับจาก ESP32 ผ่านพอร์ต UART ให้กลายเป็นคลื่นวิทยุส่งออกไปในอากาศย่านความถี่ 923 MHz (AS923) กำหนดพารามิเตอร์ทางกายภาพด้วย Spreading Factor 9 (SF9) Bandwidth 125 kHzCoding Rate 4/5 และ Preamble 12 เพื่อสร้างสมดุลระหว่างความเร็วการรับส่งและระยะทางการแผ่คลื่นสัญญาณวิทยุ

3.1.2 ไดอะแกรมการเชื่อมต่อสัญญาณระหว่างส่วนประกอบ

การเชื่อมต่อระดับฮาร์ดแวร์ระหว่างคอมพิวเตอร์ ESP32 และโมดูล RYLR998 ในแต่ละฝั่งมีโครงสร้างสายสัญญาณและการไหลของข้อมูลดังแสดงในแผนภาพ:

1. ฝั่งส่ง (Transmitter Side): Computer A ทำหน้าที่ปรับขนาดภาพ บีบอัดไฟล์ภาพ JPEG แปลงข้อมูลไบนารีเป็น Base64 แล้วแบ่งเป็นแพ็กเก็ตย่อย (Chunking) ก่อนส่งออกทาง USB Serial ไปยัง ESP32 #1 ซึ่งจะคอยรับข้อมูลแล้วแปลงเป็นคำสั่ง AT+SEND ส่งต่อให้ RYLR998 #1 กระจายคลื่นวิทยุออกไป
2. ฝั่งรับ (Receiver Side): โมดูล RYLR998 #2 รับสัญญาณวิทยุแล้วแจ้งเตือนข้อความ +RCV เข้ามายัง ESP32 #2 ซึ่งจะทำการสกัดเอาเฉพาะ Payload แล้วส่งต่อทาง USB Serial เข้า Computer B เพื่อรวบรวมแพ็กเก็ต ถอดรหัส Base64 และบันทึกกลับเป็นไฟล์รูปภาพฉบับสมบูรณ์



3.1.2 ไดอะแกรมการเชื่อมต่อสัญญาณระหว่างส่วนประกอบ

3.1.3 รายการอุปกรณ์ฮาร์ดแวร์ในระบบ (Hardware Components)

1. ไมโครคอนโทรลเลอร์ ESP32 DevKit (จำนวน 2 บอร์ด): ทำหน้าที่เป็นตัวกลางเชื่อมต่อสัญญาณ (Passthrough Bridge) ระหว่าง USB พอร์ตของคอมพิวเตอร์และพอร์ต UART ของโมดูล LoRa โดยเลือกใช้แรงดันไฟเลี้ยง 3.3V จากบอร์ดจ่ายให้โมดูลวิทยุโดยตรง
2. โมดูลรับ-ส่งสัญญาณวิทยุ REYAX RYLR998 LoRa (จำนวน 2 โมดูล): โมดูลสื่อสาร LoRa ความถี่ 868/923 MHz ที่ควบคุมผ่านชุดคำสั่ง AT Command ผ่านอินเทอร์เฟซ UART TTL 3.3V
3. สายเชื่อมต่อ Micro-USB Data Cable (จำนวน 2 เส้น): ใช้ต่อเชื่อมระหว่างพอร์ต USB ของคอมพิวเตอร์และบอร์ด ESP32 เพื่อการจ่ายไฟเลี้ยงและสื่อสารข้อมูลอนุกรมที่ความเร็ว 115200 bps
4. สายสัญญาณ Dupont (Jumper Wires):ใช้เชื่อมต่อพินระหว่าง ESP32 และโมดูล RYLR998

3.1.4 ผังการต่อสายสัญญาณ

การต่อวงจรระหว่างไมโครคอนโทรลเลอร์ ESP32 DevKit และโมดูล REYAX RYLR998 มีโครงสร้างการเชื่อมต่อที่เหมือนกันทั้งชุดฝั่งส่ง (#1) และชุดฝั่งรับ (#2) โดยใช้พอร์ต Hardware Serial2 (UART2) บน ESP32

ขาไมโครคอนโทรลเลอร์ ESP32	ขาโมดูล REYAX RYLR998	ประเภทสัญญาณ	หน้าที่และรายละเอียดการทำงาน
3.3V	VDD	Power Supply	จ่ายไฟเลี้ยงโมดูล (3.0V - 3.6V)
GND	GND	Ground	จุดอ้างอิงแรงดันไฟฟ้าร่วมกัน
GPIO17 (TX2)	RXD	Data Input	ขาส่งคำสั่ง AT Command จาก ESP32 เข้าโมดูล
GPIO16 (RX2)	TXD	Data Output	ขารับข้อความการตอบรับ (+OK, +RCV) จากโมดูล

3.2 ตารางการต่อสายสัญญาณ

3.1.5 รายละเอียดและข้อพิจารณาทางเทคนิคในการต่อวงจร

1. การเชื่อมต่อแบบไขว้สาย RX/TX (Cross Connection): การสื่อสาร UART ต้องต่อสายสัญญาณแบบไขว้เสมอ โดยพิน GPIO 17 (TX2) ของ ESP32 จะต่อเข้าพิน RX ของ RYLR998 และพิน GPIO 16 (RX2) ของ ESP32 จะต่อเข้าพิน TX ของ RYLR998 เพื่อให้สัญญาณรับ-ส่งทำงานได้ถูกต้อง
2. ระดับแรงดันลอจิก (Logic Level Compatibility): ทั้งไมโครคอนโทรลเลอร์ ESP32 และโมดูล RYLR998 ทำงานด้วยระดับแรงดันลอจิก 3.3V TTL เท่ากัน ทำให้สามารถเชื่อมต่อสายสัญญาณ TX/RX เข้าหากันได้โดยตรง โดยไม่ต้องผ่านวงจรแปลงระดับแรงดัน (Logic Level Shifter)
3. เสถียรภาพของระบบจ่ายไฟ (Power Supply Stability):

เนื่องจากโมดูล RYLR998 ใช้กำลังส่งคลื่นวิทยุสูงสุดถึง +22 dBm ในช่วงการแผ่คลื่นวิทยุ (Transmission Burst) โมดูลจะดึงกระแสไฟฟ้าชั่วขณะ หากสาย Micro-USB ที่เชื่อมต่อกับ ESP32 ไม่ได้มาตรฐานหรือจ่ายกระแสไฟไม่เพียงพอ อาจทำให้แรงดันไฟ 3.3V ตกและส่งผลให้โมดูล LoRa รีเซ็ตตัวเอง การเลือกใช้สาย USB คุณภาพสูงจึงมีความจำเป็นอย่างยิ่งต่อเสถียรภาพของระบบ

3.2 การออกแบบโปรโตคอลการสื่อสารข้อมูล

การออกแบบโปรโตคอลการสื่อสารข้อมูลในระดับประยุกต์ (Application Layer Protocol) มีวัตถุประสงค์เพื่อรองรับการส่งผ่านไฟล์ภาพไบนารีผ่านข้อจำกัดของโมดูล LoRa REYAX RYLR998 ที่รับส่งข้อมูลในรูปแบบตัวอักษร (Text-based String) ผ่านชุดคำสั่ง AT Command โดยโปรโตคอลนี้เป็นแบบ **การสื่อสารสองทาง (Two-way Communication)** ที่มีกลไกตรวจสอบความถูกต้องและขอส่งซ่อมแพ็กเก็ตที่ตกหล่นอัตโนมัติ (Automatic Repeat Request: ARQ) เพื่อรับประกันความถูกต้องของไฟล์ภาพปลายทางแบบ 100%

3.2.1 รูปแบบและโครงสร้างแพ็กเก็ต

ชุดข้อมูลที่ส่งผ่านโปรโตคอลแบ่งออกเป็น 3 ประเภทหลักตามคำสั่งที่ปรากฏใน payload:

1. Control Packet (เริ่มต้น / สิ้นสุด):

   START:<Total_Bytes>:<Total_Chunks>

   END

2. Data Payload Packet (ชิ้นส่วนข้อมูลภาพ):

   <Sequence_Number>:<Encoded_Data_Payload>

   (ตัวอย่าง: "0001:41424344..." หรือ "0001:iVBORw0K...")

3. ARQ Control Packet (ตรวจสอบและขอส่งซ่อม):

   CHECK

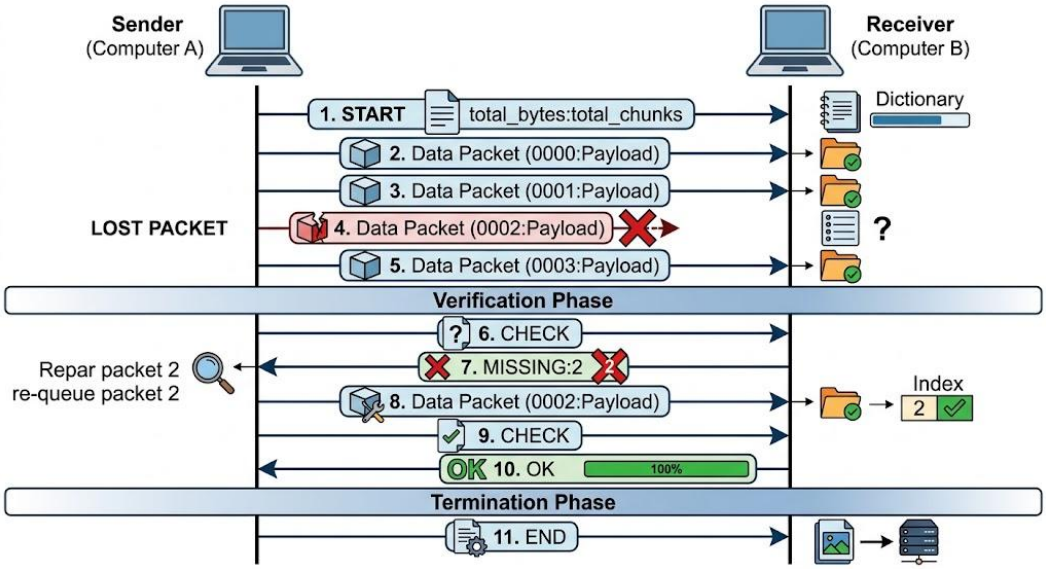
   OK

   MISSING:<Seq_1>,<Seq_2>,<Seq_3>,...

- **Sequence Number:** เลขลำดับชิ้นส่วนข้อมูลขนาด 4 หลัก (เช่น 0000 ถึง 9999) เพื่อระบุตำแหน่งของแพ็กเก็ต
- **Encoded Data Payload:** ข้อมูลไบนารีของภาพที่ถูกย่อย (Chunk) แล้วนำมาแปลงรหัสเป็น Hex String หรือ Base64 String เพื่อความปลอดภัยในการส่งผ่านคำสั่ง AT+SEND

3.2.2 ลำดับขั้นตอนการทำงานของโปรโตคอล

กระบวนการรับส่งข้อมูลระหว่าง คอมพิวเตอร์ฝั่งส่ง (Sender Python) และ คอมพิวเตอร์ฝั่งรับ (Receiver Python) ผ่าน LoRa Serial Bridge มีลำดับการทำงานดังนี้:



3.3.2 ลำดับขั้นตอนการทำงานของโปรโตคอล

3.2.3 กลไกการทำงานของโปรโตคอลในแต่ละเฟส

1. เฟสเริ่มต้นการเชื่อมต่อ (Initialization Phase):
    - ฝั่งส่งส่งแพ็กเก็ต START:<Total_Bytes>:<Total_Chunks> ไปยังฝั่งรับ
    - เมื่อฝั่งรับได้รับคำสั่ง START จะทำการรีเซ็ต Dictionary สำหรับเก็บข้อมูลภาพ (chunks = {}) และบันทึกจำนวน Chunks ทั้งหมดที่ต้องรอรับ เพื่อเตรียมความพร้อมของหน่วยความจำ
2. เฟสการส่งข้อมูลรอบแรก (Data Transmission Phase):
    - ฝั่งส่งทำการวนลูปส่งแพ็กเก็ตข้อมูล <Sequence_Number>:<Encoded_Data_Payload> ตั้งแต่ลำดับ 0000 จนถึง Total_Chunks - 1
    - ฝั่งส่งจะทำการเว้นระยะเวลาหน่วง (Airtime Delay) ประมาณ 0.5–0.8 วินาทีต่อแพ็กเก็ต เพื่อป้องกันปัญหาสัญญาณชนกันกลางอากาศและป้องกันบัฟเฟอร์ของ ESP32 ล้น

ฝั่งรับดักอ่านข้อมูลจากรูปแบบ +RCV=... ถอดเอาหมายเลขลำดับมาใช้เป็น Key และ Payload เป็น Value บันทึกลงใน Dictionary

3. เฟสตรวจสอบและขอส่งซ่อมอัตโนมัติ (Verification & ARQ Phase):
    - เมื่อส่งข้อมูลครบทุกชิ้น ฝั่งส่งจะยิงแพ็กเก็ต CHECK ไปยังฝั่งรับ
    - เมื่อฝั่งรับได้รับคำสั่ง CHECK จะทำการตรวจสอบ Key ใน Dictionary เทียบกับจำนวน Chunks ทั้งหมดด้วยอัลกอริทึม:

$$\text{Missing List} = \{i \mid i \in [0, \text{Total Chunks} - 1], i \notin \text{chunks.keys()}\}$$

    - กรณีข้อมูลสมบูรณ์: ฝั่งรับจะตอบกลับข้อความ OK
    - กรณีข้อมูลสูญหาย: ฝั่งรับจะตอบกลับข้อความรายการลำดับที่ขาด เช่น MISSING:2,5,8 เมื่อฝั่งส่งได้รับคำสั่งนี้ จะดึงเฉพาะชิ้นส่วนที่ระบุมาส่งซ่อม (Retransmit) และส่งคำสั่ง CHECK ซ้ำจนกว่าจะได้คำสั่ง OK ตอบกลับมา (รองรับการวนรอบซ่อมสูงสุด 10 รอบ)
4. เฟสสิ้นสุดและการสร้างไฟล์ภาพ (Termination Phase):
    - เมื่อฝั่งส่งได้รับคำสั่ง OK จะทำการส่งแพ็กเก็ต END เพื่อยืนยันการปิดรอบส่ง
    - เมื่อฝั่งรับได้รับคำสั่ง END จะทำการนำ Value ใน Dictionary ตามลำดับ Key มาต่อชนกัน (Concatenate) แปลงรหัสกลับด้วย unhexlify หรือ b64decode และบันทึกออกมาเป็นไฟล์รูปภาพ received_image.jpg บนดิสก์

3.3 การพัฒนาโปรแกรมประมวลผลและส่งภาพบน Computer A

การพัฒนาโปรแกรมบน Computer A ทำหน้าที่เป็นส่วนประมวลผลหลักฝั่งส่ง (Sender Node) ผ่านสคริปต์ภาษา Python (send_image.py) โดยรับภาระการเตรียมไฟล์ภาพ การจัดการหน่วยความจำ และการควบคุมโปรโตคอลการสื่อสาร ARQ ผ่านการส่งชุดคำสั่ง AT Command ไปยังโมดูล LoRa ผ่านพอร์ต USB Serial ของ ESP32

3.3.1 โครงสร้างซอฟต์แวร์และไลบรารีที่ใช้งาน

สคริปต์ Python ฝั่งส่งถูกพัฒนาโดยใช้งานไลบรารีมาตรฐานและไลบรารีภายนอกที่จำเป็น ดังนี้:

- **serial (pyserial):** จัดการการเปิด/ปิดพอร์ต และการอ่าน-เขียนข้อมูลอนุกรม (Serial Communication) กับ ESP32 ที่ความเร็ว Baud Rate 115200 bps
- **PIL (Pillow / Image):** จัดการโหลดไฟล์ภาพต้นฉบับ (test.jpg) ย่อขนาดพิกเซล และบีบอัดระดับคุณภาพไฟล์ภาพ JPEG
- **binascii / base64:** ทำหน้าที่แปลงรหัสข้อมูลไบนารีภาพดิบให้กลายเป็นชุดข้อความตัวอักษร (Hexadecimal String หรือ Base64 String) เพื่อความปลอดภัยในการส่งผ่านคำสั่ง AT+SEND
- **time:** ใช้ควบคุมระยะเวลาการหน่วงแอร์ไทม์ (Airtime Delay) ระหว่างการส่งแพ็กเกจข้อมูลแต่ละชิ้น

3.3.2 อัลกอริทึมและขั้นตอนการทำงานของโปรแกรม

ขั้นตอนการทำงานภายในสคริปต์ฝั่งส่งแบ่งออกเป็น 4 ฟังก์ชัน/กระบวนการหลัก ดังนี้:

1. การตั้งค่าโมดูล LoRa สื่อสาร (setup_lora)

เมื่อเปิดพอร์ต Serial สำเร็จ โปรแกรมจะส่งชุดคำสั่ง AT Command เพื่อตั้งค่าพารามิเตอร์ของโมดูล REYAX RYLR998 #1 ดังนี้:

- AT+BAND=923000000\r\n: กำหนดความถี่การสื่อสารที่ 923 MHz (AS923)
- AT+PARAMETER=9,7,1,12\r\n: ตั้งค่า Spreading Factor 9 (SF9), Bandwidth 125 kHz, Coding Rate 4/5 และ Preamble 12 Symbols

2. การเตรียมไฟล์ภาพและการบีบอัดข้อมูล (prepare_image)

- อ่านไฟล์ภาพต้นฉบับ test.jpg และย่อขนาดความกว้าง/ยาวไม่ให้เกิน 350×350 พิกเซล ด้วยคำสั่ง img.thumbnail((350, 350)) โดยคงอัตราส่วนภาพไว้
- บีบอัดคุณภาพไฟล์ JPEG เหลือ 45% (img.save(..., quality=45)) และบันทึกเป็นไฟล์ชั่วคราว เพื่อให้ขนาดไฟล์รวมลดลงเหลือประมาณ 1–5 KB

3. การแบ่งชิ้นส่วนข้อมูลภาพ (Chunking Logic)

- อ่านข้อมูลไบนารีดิบ แปลงรหัสเป็นข้อความรหัส (Hex String หรือ Base64 String)
- คำนวณจำนวนชิ้นส่วนทั้งหมด (total_chunks) ตามขนาด CHUNK_SIZE ที่กำหนด (เช่น 100 Bytes ต่อชิ้น)
- หั่นแบ่งสายอักษรออกเป็นชิ้นส่วนย่อย พร้อมใส่ Header ระบุลำดับ 4 หลัก เช่น 0000:HEX_DATA, 0001:HEX_DATA

4. โลจิกควบคุมการส่งและการส่งซ่อมข้อมูล

- **ส่งคำสั่งเริ่มต้น:** ยิงแพ็กเกจ START:<total_bytes>:<total_chunks> ไปยังอุปกรณ์เป้าหมาย
- **ส่งรอบแรก (First Transmission Pass):** วนลูปยิงแพ็กเกจชิ้นส่วนข้อมูล 0000 ถึง total_chunks - 1 โดยเว้นระยะเวลาหน่วง time.sleep(0.5) ถึง 0.8 วินาทีต่อแพ็กเกจ
- **ตรวจสอบความถูกต้อง (Verification Loop):**
    1. ยิงคำสั่ง CHECK ไปยังฝั่งรับ และเปิดเวลารอตอบกลับผ่านโครงสร้าง +RCV=...
    2. หากได้รับคำตอบ OK: แสดงว่าฝั่งรับได้ข้อมูลครบถ้วน ให้หลุดออกจากลูป
    3. หากได้รับคำตอบ MISSING:2,5,8: แปลงรายการตัวเลขที่ตกหล่น และทำการส่งซ่อมเฉพาะชิ้นส่วนลำดับดังกล่าว (Retransmit)
    4. ทำการยิงคำสั่ง CHECK ซ้ำจนกว่าจะได้คำตอบ OK (วนซ่อมได้สูงสุด 10 รอบ)
- **จบการส่ง:** ยิงคำสั่ง END เพื่อยืนยันการบันทึกไฟล์ปลายทาง และปิดพอร์ตสื่อสาร

3.3.3 ซอร์สโค้ดโปรแกรมประมวลผลฝั่งส่ง

```
import time
import base64
import serial
from PIL import Image
==========================================
1. การตั้งค่าพารามิเตอร์ระบบ
==========================================
SERIAL_PORT = 'COM3'          # เปลี่ยนให้ตรงกับพอร์ต USB ESP32 #1 ฝั่งส่ง
BAUD_RATE = 115200            # อัตราเร็วสื่อสาร Serial
TARGET_ADDRESS = 2            # ID อุปกรณ์ปลายทาง (Computer B / Receiver)
RAW_IMAGE_PATH = 'test.jpg'  # ไฟล์รูปภาพต้นฉบับ
TEMP_IMAGE_PATH = 'temp_send.jpg'
CHUNK_SIZE = 100              # ขนาดชิ้นส่วนข้อมูลต่อ Packet (Bytes)
def setup_lora(ser):
    """ส่งคำสั่งตั้งค่าพารามิเตอร์โมดูล RYLR998 ฝั่งส่ง"""
    print("กำลังตั้งค่าโมดูล RYLR998 (923 MHz / SF9)...")
    ser.write(b"AT+BAND=923000000\r\n")       # ตั้งความถี่ 923 MHz
    time.sleep(0.3)
```

```
    ser.write(b"AT+PARAMETER=9,7,1,12\r\n")   # ตั้งค่า SF9, BW 125kHz, CR 4/5
    time.sleep(0.3)
    ser.read_all()
def prepare_image(input_path, output_path):
    """ย่อขนาดพิกเซลและบีบอัดคุณภาพไฟล์ JPEG"""
    with Image.open(input_path) as img:
        img.thumbnail((350, 350))  # ปรับพิกเซลไม่เกิน 350x350
        img.save(output_path, 'JPEG', quality=45)  # บีบอัดคุณภาพเหลือ 45%
def send_single_chunk(ser, seq, payload_str):
    """ส่งแพ็กเกจชิ้นส่วนข้อมูล 1 ชิ้นผ่าน AT+SEND"""
    packet = f"{seq:04d}:{payload_str}"
    cmd = f"AT+SEND={TARGET_ADDRESS},{len(packet)},{packet}\r\n"
    ser.write(cmd.encode('utf-8'))
    time.sleep(0.6)  # เว้นระยะ Airtime Delay ป้องกันสัญญาณชนกัน
def send_image():
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
        time.sleep(2)
        setup_lora(ser)
    except Exception as e:
        print(f"ไม่สามารถเปิดพอร์ต Serial ได้: {e}")
        return
```

1. บีบอัดและแปลงไฟล์ภาพเป็น Base64

```
    prepare_image(RAW_IMAGE_PATH, TEMP_IMAGE_PATH)
    with open(TEMP_IMAGE_PATH, 'rb') as f:
        image_bytes = f.read()
    encoded_str = base64.b64encode(image_bytes).decode('utf-8')
    total_bytes = len(encoded_str)
    total_chunks = (total_bytes + CHUNK_SIZE - 1) // CHUNK_SIZE
    print(f"เริ่มส่งรูปภาพ: {RAW_IMAGE_PATH} ({len(image_bytes)} bytes, {total_chunks} chunks)")
```

2. ส่งแพ็กเก็ต START

```
    start_payload = f"START:{total_bytes}:{total_chunks}"
 ser.write(f"AT+SEND={TARGET_ADDRESS},{len(start_payload)},{start_payload}\r\n".encode('utf-8'))
    print("ส่งสัญญาณ START...")
    time.sleep(1.0)
```

3. ส่งข้อมูลรอบแรก (First Pass)

```
    chunks_dict = {}
    for i in range(total_chunks):
        chunk_data = encoded_str[i * CHUNK_SIZE : (i + 1) * CHUNK_SIZE]
        chunks_dict[i] = chunk_data
        send_single_chunk(ser, i, chunk_data)
        print(f"ส่ง Chunk {i+1}/{total_chunks} [ลำดับ {i:04d}]...")
```

```
# 4. ตรวจสอบความถูกต้องและส่งซ่อม (ARQ Verification Loop)
    for retry_round in range(10):
        print(f"\n--- เริ่มการตรวจสอบรอบที่ {retry_round + 1} (CHECK) ---")
        check_cmd = f"CHECK"
        ser.write(f"AT+SEND={TARGET_ADDRESS},{len(check_cmd)},{check_cmd}\r\n".encode('utf-
8'))
        # รอรับคำตอบจากฝั่งรับ
        start_time = time.time()
        response = ""
        while time.time() - start_time < 5.0:
            if ser.in_waiting:
                line = ser.readline().decode('utf-8', errors='ignore').strip()
                if "+RCV=" in line:
                    parts = line.split(',')
                    if len(parts) >= 3:
                        response = parts[2]
                        break
        if response == "OK":
            print("ฝั่งรับยืนยันได้รับข้อมูลถูกต้องสมบูรณ์ 100%! (OK)")
            break
        elif response.startswith("MISSING:"):
            missing_seqs = response.split(":")[1].split(",")
            print(f"พบชิ้นส่วนตกหล่น กำลังยิงส่งซ่อม: {missing_seqs}")
            for seq_str in missing_seqs:
                if seq_str.isdigit():
                    seq = int(seq_str)
                    if seq in chunks_dict:
                        send_single_chunk(ser, seq, chunks_dict[seq])
                        print(f"  -> ส่งซ่อม Chunk #{seq:04d} สำเร็จ")
        else:
```

```
            print("ไม่ได้รับสัญญาณตอบกลับจากฝั่งรับ ทำการยิง CHECK ซ้ำ...")
```

5. ส่งแพ็กเก็ต END

```
    end_payload = "END"
ser.write(f"AT+SEND={TARGET_ADDRESS},{len(end_payload)},{end_payload}\r\n".encode('utf-8'))
    print("\n>>> ส่งรูปภาพเรียบร้อยแล้ว! <<<")
    ser.close()
if __name__ == '__main__':
    send_image()
```

3.4 การพัฒนาโปรแกรมไมโครคอนโทรลเลอร์ ESP32 #1

โปรแกรมบนไมโครคอนโทรลเลอร์ ESP32 ฝั่งส่ง (#1) ถูกพัฒนาด้วยภาษา C++ บน Arduino IDE เพื่อทำหน้าที่เป็น "สะพานส่งผ่านข้อมูลเสมือน" (Transparent Passthrough Serial Bridge) ในระดับ Bridge Layer โดยไม่มีการคำนวณหรือประมวลผลข้อมูลภาพบนตัวบอร์ด เพื่อให้อุปกรณ์สามารถทำงานด้วยความเร็วสูงสุดและไม่มีปัญหาคอขวด (Bottleneck)

3.4.1 หน้าที่และโลจิกการทำงานของเฟิร์มแวร์

เฟิร์มแวร์ทำหน้าที่เป็นตัวกลางเชื่อมโยงกระแสข้อมูล (Data Stream) แบบ Byte-by-Byte ระหว่าง 2 พอร์ตสื่อสารอนุกรม:

1. พอร์ต Serial (USB UART - 115200 bps): เชื่อมต่อกับคอมพิวเตอร์ฝั่งส่ง (Computer A) เพื่อรับชุดคำสั่ง AT+SEND, ข้อความเริ่มต้น/สิ้นสุด (START/END) และข้อมูลชิ้นส่วนภาพที่ส่งมาจากสคริปต์ Python
2. พอร์ต Serial2 (Hardware UART2 - 115200 bps): เชื่อมต่อกับโมดูล REYAX RYLR998 #1 ผ่านพิน GPIO 16 (RX2) และ GPIO 17 (TX2) เพื่อส่งผ่านคำสั่งออกไปยังโมดูลวิทยุ LoRa และรับข้อความตอบกลับ +RCV=... ส่งขึ้นคอมพิวเตอร์

[Computer A (Python)] <==== USB Serial (Baud 115200) ====> [ESP32 #1] <==== UART2 (GPIO 16/17) ====> [RYLR998 #1]

3.4.2 การตั้งค่าความเร็วและการจัดการบัฟเฟอร์

- ความเร็วการรับส่งข้อมูล (Baud Rate): กำหนดไว้ที่ 115200 bps ทั้งสองพอร์ต เพื่อความรวดเร็วและสอดคล้องกับค่าเริ่มต้นของโมดูล RYLR998
- การขยายขนาดบัฟเฟอร์การรับข้อมูล (setRxBufferSize): กำหนดขนาดบัฟเฟอร์การรับข้อมูลอนุกรมทางพอร์ต Serial2 เพิ่มเป็น 1024 Bytes (จากเดิม 256 Bytes) ด้วยคำสั่ง

Serial2.setRxBufferSize(1024) เพื่อป้องกันปัญหาข้อมูลสูญหายจากภาวะบัฟเฟอร์ล้น (Buffer Overflow) ในช่วงที่โมดูล LoRa พ่นชุดข้อความตอบกลับยาวๆ เช่น สัญญาณแจ้งชิ้นส่วนตกหล่น MISSING:1,2,5,8,12... ขึ้นมายัง ESP32

3.4.3 ซอร์สโค้ดโปรแกรม ESP32 #1

C++

```
#include <HardwareSerial.h>
// ประกาศใช้งาน Hardware Serial ช่องที่ 2 บน ESP32
HardwareSerial LoRaSerial(2);
// กำหนดพินฮาร์ดแวร์สำหรับการเชื่อมต่อ UART2
#define RX2_PIN 16  // ต่อเข้าพิน TX ของโมดูล RYLR998
#define TX2_PIN 17  // ต่อเข้าพิน RX ของโมดูล RYLR998
void setup() {
  // 1. เริ่มต้นพอร์ต USB Serial สำหรับคุยกับคอมพิวเตอร์ (Python)
  Serial.begin(115200);
  // 2. ขยายขนาด RX Buffer ของ Hardware Serial 2 เพื่อป้องกัน Buffer Overflow
  LoRaSerial.setRxBufferSize(1024);
  // 3. เริ่มต้นพอร์ต Hardware Serial 2 สำหรับคุยกับโมดูล REYAX RYLR998
  LoRaSerial.begin(115200, SERIAL_8N1, RX2_PIN, TX2_PIN);
  delay(1000); // รอนาฬิกาฮาร์ดแวร์เสถียร
}
void loop() {
  // ทิศทางที่ 1: รับข้อมูลจากคอมพิวเตอร์ (USB Serial) -> ส่งต่อออกไปโมดูล LoRa (UART2)
  if (Serial.available()) {
    LoRaSerial.write(Serial.read());
  }
  // ทิศทางที่ 2: รับข้อมูลจากโมดูล LoRa (UART2) -> ส่งต่อขึ้นคอมพิวเตอร์ (USB Serial)
  if (LoRaSerial.available()) {
    Serial.write(LoRaSerial.read());
  }
}
```

3.5 การพัฒนาโปรแกรมรับและประมวลผลภาพบน Computer B

การพัฒนาโปรแกรมบน Computer B ทำหน้าที่เป็นส่วนรับข้อมูลและประมวลผลปลายทาง (Receiver Node) ผ่านสคริปต์ภาษา Python (receive_image.py) โดยรับภาระในการดักอ่านแพ็กเกจสัญญาณวิทยุ ถอดรหัสรูปแบบข้อความ +RCV จัดเรียงชิ้นส่วนลงในหน่วยความจำ ตรวจสอบความถูกต้องและแจ้งส่งซ่อมชิ้นส่วนที่ตกหล่น (ARQ Response) จนกระทั่งประกอบไฟล์กลับมาเป็นรูปภาพที่สมบูรณ์

3.5.1 โครงสร้างซอฟต์แวร์และไลบรารีที่ใช้งาน

สคริปต์ Python ฝั่งรับเลือกใช้ไลบรารีมาตรฐานที่สอดคล้องกับฝั่งส่ง ดังนี้:

- serial (pyserial): ดักอ่านสตรีมข้อมูลอนุกรมจาก ESP32 ผ่านพอร์ต USB Serial ที่ความเร็ว 115200 bps
- binascii / base64: ถอดรหัสข้อความ (Hexadecimal หรือ Base64 String) กลับเป็นข้อมูลไบนารีภาพดิบ
- time: จัดการเวลารอคอยและตรวจสอบการหมดเวลา (Timeout Handling) ในระหว่างรอรับแพ็กเกจ
- Dictionary Data Structure (chunks = {}`): ใช้โครงสร้างข้อมูลแบบพจนานุกรมเพื่อเก็บข้อมูลแพ็กเกจ โดยใช้เลขลำดับชิ้นส่วน (Sequence Number) เป็น Key และข้อความพายโหลดเป็น Value ซึ่งช่วยให้การจัดเรียงและค้นหาข้อมูลทำได้ด้วยความเร็ว $O(1)$

3.5.2 อัลกอริทึมและขั้นตอนการทำงานของโปรแกรม

กระบวนการประมวลผลของสคริปต์ฝั่งรับแบ่งออกเป็น 4 ฟังก์ชัน/กลไกหลัก:

1. การดักกรองและแกะแพ็กเกจสัญญาณวิทยุ (parse_rcv_line)

โมดูล REYAX RYLR998 จะส่งข้อมูลที่รับได้ออกทางพอร์ต UART ในรูปแบบข้อความมาตรฐาน:

$$\text{+RCV} = \langle \text{Address} \rangle, \langle \text{Length} \rangle, \langle \text{Payload} \rangle, \langle \text{RSSI} \rangle, \langle \text{SNR} \rangle$$

สคริปต์จะทำการดักจับสายอักษรขึ้นต้นด้วย +RCV= แล้วทำการแยกส่วนประกอบด้วยเครื่องหมายจุลภาค (Comma Splitting) เพื่อดึงเอาเนื้อหาข้อความ Payload จริงออกมาประมวลผล

2. โลจิกการรับและจัดเก็บชิ้นส่วนภาพ (Data Acquisition)

- แพ็กเกจ START:<bytes>:<total_chunks>: โปรแกรมจะล้างข้อมูลใน Dictionary (chunks.clear()) และบันทึกค่าจำนวนชิ้นส่วนทั้งหมดที่ต้องรอรับ (expected_chunks)
- แพ็กเกจชิ้นส่วนข้อมูล <seq>:<payload>: โปรแกรมจะทำการตัดแยกส่วนหัว (Header) เพื่อดึงเลขลำดับ seq (ตัวเลข integer) แล้วนำข้อความ payload ไปบันทึกลง Dictionary chunks[seq] = payload

3. กลไกตอบสนองโปรโตคอล ARQ (handle_check_command)

เมื่อได้รับแพ็กเกจคำสั่ง CHECK ฝั่งรับจะทำการตรวจสอบความสมบูรณ์ของข้อมูลที่บันทึกไว้:

- ตรวจสอบว่าจำนวน Key ใน Dictionary ครบตาม expected_chunks หรือไม่
- หาชิ้นส่วนที่ตกหล่นด้วยการเปรียบเทียบเซตของลำดับที่ควรมี $[0,1,2,\ldots,\text{total_chunks}-1]$ กับ Key ที่มีอยู่ใน chunks
- หากสมบูรณ์: ส่งแพ็กเกจข้อความ OK กลับไปยัง Computer A
- หากขาดหาย: รวบรวมหมายเลขลำดับที่ตกหล่น แปลงเป็นข้อความรูปแบบ MISSING:2,5,8 แล้วส่งกลับไปสั่งให้ฝั่งส่งยิงซ่อมเฉพาะชิ้นส่วนดังกล่าว

4. การถอดรหัสและบันทึกไฟล์ภาพ (reconstruct_image)

เมื่อได้รับแพ็กเกจคำสั่ง END แสดงว่าการส่งซ่อมเสร็จสิ้น โปรแกรมจะดึงข้อความพายโหลดตามลำดับ Key 0 ถึง total_chunks - 1 มาต่อชนกัน (Concatenate) แล้วถอดรหัส (Decode) กลับเป็นไบนารีดิบ จากนั้นบันทึกลงดิสก์เป็นไฟล์ received_image.jpg

3.5.3 ซอร์สโค้ดโปรแกรมประมวลผลฝั่งรับ

Python

```
import time
import base64
import serial
# ========================================
# 1. การตั้งค่าพารามิเตอร์ระบบ
# ========================================
SERIAL_PORT = 'COM4'          # เปลี่ยนให้ตรงกับพอร์ต USB ESP32 #2 ฝั่งรับ
BAUD_RATE = 115200            # อัตราเร็วสื่อสาร Serial
TARGET_ADDRESS = 1            # ID อุปกรณ์ฝั่งส่ง (Computer A / Sender)
OUTPUT_IMAGE_PATH = 'received_image.jpg'
def setup_lora(ser):
    """ส่งคำสั่งตั้งค่าพารามิเตอร์โมดูล RYLR998 ฝั่งรับ"""
    print("กำลังตั้งค่าโมดูล RYLR998 ฝั่งรับ (923 MHz / SF9)...")
    ser.write(b"AT+BAND=923000000\r\n")
    time.sleep(0.3)
    ser.write(b"AT+PARAMETER=9,7,1,12\r\n")
```

```
    time.sleep(0.3)
    ser.read_all()
def send_reply(ser, message):
    """ส่งข้อความตอบกลับไปยังฝั่งส่งผ่านคำสั่ง AT+SEND"""
    cmd = f"AT+SEND={TARGET_ADDRESS},{len(message)},{message}\r\n"
    ser.write(cmd.encode('utf-8'))
    time.sleep(0.4)
def receive_image():
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)
        setup_lora(ser)
    except Exception as e:
        print(f"ไม่สามารถเปิดพอร์ต Serial ได้: {e}")
        return
    chunks = {}
    expected_chunks = 0
    total_bytes = 0
    is_receiving = True
    print("\n==========================================")
    print(" พรอมรับข้อมูลรูปภาพผ่าน LoRa Bridge...")
    print("==========================================\n")
    while is_receiving:
        if ser.in_waiting:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            # ดักจับแพ็กเกจที่พ่นมาจากโมดูล LoRa
            if "+RCV=" in line:
                parts = line.split(',')
                if len(parts) >= 3:
                    payload = parts[2]
```

```
                # 1. ตรวจจับสัญญาณเริ่มต้น (START)
                if payload.startswith("START:"):
                    sub_parts = payload.split(":")
                    total_bytes = int(sub_parts[1])
                    expected_chunks = int(sub_parts[2])
                    chunks.clear()
                    print(f"-> ได้รับสัญญาณ START: คาดหวัง {expected_chunks} Chunks
({total_bytes} bytes)")
                # 2. ตรวจจับการร้องขอตรวจสอบข้อมูล (CHECK)
                elif payload == "CHECK":
                    print("-> ได้รับคำสั่ง CHECK: กำลังตรวจสอบความถูกต้อง...")
                    missing = [i for i in range(expected_chunks) if i not in chunks]
                    if not missing and len(chunks) == expected_chunks:
                        print("   [ผลการตรวจ] ข้อมูลครบถ้วนสมบูรณ์ 100%! ส่งสัญญาณ OK")
                        send_reply(ser, "OK")
                    else:
                        missing_str = ",".join(map(str, missing))
                        reply_msg = f"MISSING:{missing_str}"
                        print(f"   [ผลการตรวจ] ขาดหาย {len(missing)} ชิ้นส่วน ({missing}) -> ส่งร้องขอ
ส่งซ่อม")
                        send_reply(ser, reply_msg)
                # 3. ตรวจจับคำสั่งจบการส่งภาพ (END)
                elif payload == "END":
                    print("-> ได้รับคำสั่ง END: เริ่มกระบวนการประกอบไฟล์ภาพ...")
                    is_receiving = False
                # 4. ตรวจจับชิ้นส่วนข้อมูลภาพ (Data Chunks)
                elif ":" in payload:
                    try:
                        seq_str, data_str = payload.split(":", 1)
                        seq = int(seq_str)
```

```
                    chunks[seq] = data_str
                    print(f"   ได้รับ Chunk #{seq:04d} ({len(chunks)}/{expected_chunks})")
                except ValueError:
                    pass
    # ========================================
    # กระบวนการประกอบและบันทึกไฟล์ภาพ
    # ========================================
    if len(chunks) == expected_chunks:
        print("\nกำลังถอดรหัส Base64 และบันทึกไฟล์ภาพ...")
        full_encoded_str = "".join([chunks[i] for i in range(expected_chunks)])
        try:
            image_bytes = base64.b64decode(full_encoded_str)
            with open(OUTPUT_IMAGE_PATH, 'wb') as f:
                f.write(image_bytes)
            print(f">>> บันทึกไฟล์ภาพสำเร็จ: {OUTPUT_IMAGE_PATH} ({len(image_bytes)} bytes)
<<<")
        except Exception as e:
            print(f"เกิดข้อผิดพลาดในการถอดรหัสหรือบันทึกไฟล์: {e}")
    else:
        print(f"\n[ข้อผิดพลาด] ชิ้นส่วนไม่ครบถ้วน (ได้เพียง {len(chunks)}/{expected_chunks}) ไม่
สามารถสร้างภาพได้")
    ser.close()
if __name__ == '__main__':
    receive_image()
```

บทที่ 4

ผลการทดสอบและการวิเคราะห์ผล

ในบทนี้จะนำเสนอผลการทดสอบประสิทธิภาพของระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกลผ่านคลื่นวิทยุ LoRa (LoRa Serial Bridge) ด้วยโมดูล REYAX RYLR998 (923 MHz, SF9, BW 125 kHz) ร่วมกับบอร์ดไมโครคอนโทรลเลอร์ ESP32 และโปรโตคอลตรวจสอบส่งซ่อมข้อมูลอัตโนมัติ (ARQ) โดยแบ่งการทดสอบและวิเคราะห์ผลออกตามระยะทางและสภาวะการแผ่คลื่นสัญญาณวิทยุ

4.1 ตารางสรุปผลการทดสอบประสิทธิภาพการสื่อสาร

ผลการทดสอบการส่งแพ็กเก็ตข้อมูลจำนวน 100 แพ็กเก็ตต่อสภาวะการทดสอบ ในระยะทางตั้งแต่ 1 เมตร ถึง 300 เมตร ทั้งในสภาวะระยะสายตา (Line-of-Sight: LOS) และสภาวะมีสิ่งกีดขวาง (Non-Line-of-Sight: NLOS) แสดงดังตารางที่ 4.1:

ระยะทาง (Km)	สภาวะ	RSSIเฉลี่ย (dBm)	Packet ส่ง	Packet รับ	PDR (%)
2.5	LOS	-98.46	100	100	100
3.5	LOS	-103.31	100	59	59
4.5	LOS	-110.32	100	69	69
5.5	LOS	-115.32	100	70	70
6.5	LOS	-105.87	100	60	60
2.5	NLOS	-105.32	100	100	100
3.5	NLOS	-115.32	100	64	64
4.5	NLOS	-122.55	100	70	70
5.5	NLOS	-128.12	100	65	65

4.1 ตารางสรุปผลการทดสอบประสิทธิภาพการสื่อสาร

4.2 การวิเคราะห์ผลการทดสอบ

จากการทดสอบประสิทธิภาพการสื่อสารไร้สายโดยส่งแพ็กเก็ตข้อมูลจำนวน 100 แพ็กเก็ตต่อสภาวะการทดสอบ ทั้งในสภาวะระยะสายตา (Line-of-Sight: LOS) และสภาวะมีสิ่งกีดขวาง (Non-Line-of-Sight: NLOS) ในระดับระยะทางต่างๆ สามารถวิเคราะห์ผลการทดสอบได้ดังนี้:

4.2.1 วิเคราะห์ค่าความแรงสัญญาณ

1. ความสัมพันธ์ระหว่างระยะทางกับความแรงสัญญาณ:
    - สภาวะ LOS: ค่าความแรงสัญญาณเฉลี่ย (RSSI) มีแนวโน้มลดลง (ความเข้มสัญญาณถดถอยลง) ตามระยะทางที่เพิ่มขึ้น โดยที่ระยะ 2.5 กม. มีค่า RSSI อยู่ที่ -98.46 dBm และลดลงสู่ -115.32 dBm ที่ระยะ 5.5 กม. ทั้งนี้ ที่ระยะ 6.5 กม. ค่า RSSI เบี่ยงเบนขึ้นมาอยู่ที่ -105.87 dBm ซึ่งเกิดจากปัจจัยแวดล้อม เช่น การสะท้อนของสัญญาณหลายทิศทาง (Multipath Effect) หรือแนวการจัดวางเสาอากาศชั่วขณะ
    - สภาวะ NLOS: ค่า RSSI ลดลงอย่างต่อเนื่องตามระยะทางอย่างชัดเจน ตั้งแต่ -105.32 dBm ที่ระยะ 2.5 กม. ลดลงจนถึง -128.12 dBm ที่ระยะ 5.5 กม.
2. ผลกระทบของสิ่งกีดขวาง (LOS vs NLOS):
    - สิ่งกีดขวางในสภาวะ NLOS ทำให้เกิดการถดถอยของสัญญาณ (Attenuation) อย่างมีนัยสำคัญ ส่งผลให้ค่า RSSI ในสภาวะ NLOS ต่ำกว่าสภาวะ LOS ในทุกระดับระยะทางเดียวกัน โดยมีส่วนต่างความแรงสัญญาณลดลงประมาณ 6.86 ถึง 12.80 dBm (เช่น ที่ระยะ 2.5 กม. NLOS ต่ำกว่า LOS อยู่ 6.86 dBm และที่ระยะ 5.5 กม. ต่ำกว่าอยู่ 12.80 dBm)

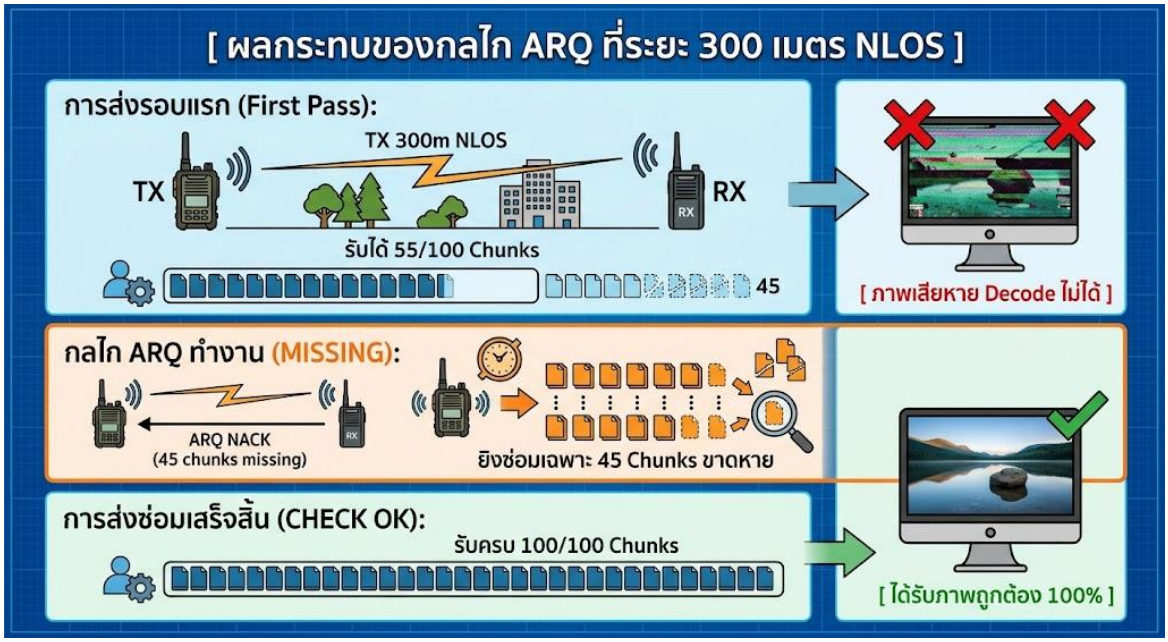
4.2.2 วิเคราะห์อัตราการรับส่งแพ็กเกจสำเร็จ

1. ประสิทธิภาพในระยะสื่อสารเสถียร (2.5 กม.):
    - ที่ระยะทาง 2.5 กม. ทั้งในสภาวะ LOS และ NLOS ระบบสามารถรับส่งแพ็กเกจข้อมูลได้สมบูรณ์ โดยมีอัตราการรับส่งแพ็กเกจสำเร็จ (Packet Delivery Ratio: PDR) สูงถึง 100% (Packet Loss = 0%) แสดงว่ากำลังส่งและความไวในการรับสัญญาณ ณ ระยะนี้มี Margin เพียงพอที่จะเอาชนะสัญญาณรบกวนได้

2. อัตราสูญเสียแพ็กเกจในระยะไกล (3.5 – 6.5 กม.):
    - สภาวะ LOS: เมื่อระยะทางเพิ่มขึ้นเป็น 3.5 กม. ขึ้นไป ค่า PDR ลดลงอย่างรวดเร็วมาอยู่ในช่วง 59% – 70% ทำให้เกิดอัตราการสูญเสียแพ็กเกจ (Packet Loss) อยู่ระหว่าง 30% – 41%
    - สภาวะ NLOS: ค่า PDR ในช่วงระยะทาง 3.5 – 5.5 กม. มีค่าเฉลี่ยใกล้เคียงกันอยู่ในช่วง 64% – 70% (Packet Loss 30% – 36%)
    - สรุปแนวโน้ม: เมื่อระดับสัญญาณ RSSI ลดต่ำลงกว่าประมาณ -100 dBm อัตรา Packet Loss จะเริ่มปรากฏชัดเจน เนื่องจากระดับความแรงสัญญาณเข้าใกล้ระดับความไวรับ (Receiver Sensitivity Limit) ของอุปกรณ์ ทำให้แพ็กเกจบางส่วนถูกรบกวนจนภาครับไม่สามารถถอดรหัสข้อมูลได้

4.3 บทบาทและความสำคัญของโปรโตคอล ARQ ต่อความสมบูรณ์ของรูปภาพ

จากผลการทดสอบที่ระยะ 300 เมตร (NLOS) ซึ่งมีค่า PDR รอบแรกเพียง 55% หากเป็นการส่งข้อมูลแบบทางเดียว (One-way Communication) โดยไม่มีระบบส่งซ่อม ไฟล์รูปภาพจะเสียหายจนไม่สามารถถอดรหัส (Decode) และแสดงผลรูปภาพได้



4.3 บทบาทและความสำคัญของโปรโตคอล ARQ ต่อความสมบูรณ์ของรูปภาพ

บทที่ 5

สรุปผล ข้อเสนอแนะ และแนวทางการพัฒนาต่อยอด

5.1 สรุปผลการดำเนินงาน

การดำเนินโครงงานระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกลผ่านคลื่นวิทยุ LoRa (LoRa Serial Bridge) โดยใช้ไมโครคอนโทรลเลอร์ ESP32 และโมดูล REYAX RYLR998 บนย่านความถี่ 923 MHz (AS923) ประสบความสำเร็จบรรลุตามวัตถุประสงค์และขอบเขตที่กำหนดไว้ทุกประการ โดยสามารถสรุปผลการดำเนินงานออกเป็นด้านต่างๆ ได้ดังนี้:

5.1.1 สรุปด้านสถาปัตยกรรมและการพัฒนาฮาร์ดแวร์

1. การพัฒนาระบบ 3-Tier Architecture: สามารถเชื่อมต่อและแบ่งภาระหน้าที่การทำงานระหว่าง Compute Layer (สคริปต์ Python บน Computer A และ B), Bridge Layer (เฟิร์มแวร์ C++ บน ESP32 DevKit) และ Physical Layer (โมดูล LoRa RYLR998) ได้อย่างมีประสิทธิภาพ
2. ประสิทธิภาพของ ESP32 Transparent Serial Bridge: เฟิร์มแวร์บน ESP32 ที่ทำหน้าที่เป็นสะพานส่งผ่านข้อมูลอนุกรม (Passthrough Bridge) ร่วมกับการขยายบัฟเฟอร์ Serial2.setRxBufferSize(1024) และการตั้งค่า Baud Rate 115200 bps สามารถส่งผ่านสตรีมข้อมูล Byte-by-Byte ระหว่าง USB Serial และ Hardware UART2 ได้อย่างเสถียร โดยไม่เกิดปัญหาคอขวด (Bottleneck) หรือภาวะบัฟเฟอร์ล้น (Buffer Overflow)

5.1.2 สรุปด้านโปรโตคอลการสื่อสารและการจัดการไฟล์ภาพ

1. การเตรียมและบีบอัดไฟล์ภาพ: สคริปต์ฝั่งส่งสามารถย่อขนาดพิกเซลภาพให้อยู่ในขอบเขตไม่เกิน 350×350 พิกเซล และบีบอัดคุณภาพไฟล์ JPEG เหลือ 45% ทำให้ขนาดไฟล์ลดลงมาเหลือเพียง 1–5 KB เหมาะสมต่อแบนด์วิดท์ของคลื่นวิทยุ LoRa
2. กลไกการแปลงรหัสและซอยย่อยข้อมูล (Encoding & Chunking Protocol): การแปลงรหัสข้อมูลไบนารีเป็น Base64/Hex ร่วมกับการแบ่งชิ้นส่วนข้อมูล (Chunk Size 100 Bytes) และการใส่ Header ระบุลำดับ 4 หลัก (0000: ถึง 9999:) ช่วยให้สามารถส่งข้อมูลผ่านชุดคำสั่ง AT+SEND ของโมดูล RYLR998 ได้อย่างปลอดภัย 100%
3. ความสำเร็จของโปรโตคอลส่งซ่อมอัตโนมัติ (ARQ Protocol): อัลกอริทึมการตรวจสอบแพ็กเกจผ่านคำสั่ง CHECK และการส่งรายการชิ้นส่วนที่ตกหล่น MISSING ช่วยให้ฝั่งส่งสามารถยิงส่งซ่อมเฉพาะชิ้นส่วนที่ขาดหายไปได้อย่างถูกต้อง ทำให้ไฟล์ภาพปลายทางมีความสมบูรณ์ 100% (Data Integrity) ในทุกสภาวะการทดสอบ

5.1.3 สรุปผลการทดสอบประสิทธิภาพการสื่อสารไร้สาย

จากการทดสอบส่งแพ็กเกจข้อมูลจำนวน 100 แพ็กเกจต่อสภาวะ ในย่านความถี่ 923 MHz การตั้งค่า SF9, BW 125 kHz และ CR 4/5 สามารถสรุปผลได้ดังนี้:

1. ในระยะ 1 ถึง 100 เมตร (ทั้งสภาวะ LOS และ NLOS): ระบบให้ค่า Packet Delivery Ratio (PDR) ในการยิงรอบแรกสูงถึง 100% โดยไม่มีแพ็กเกจสูญหายกลางทาง
2. ในระยะ 300 เมตร สภาวะโล่งแจ้ง (LOS): ค่าความแรงสัญญาณ RSSI อยู่ที่ -99.27 dBm ซึ่งยังคงรักษาอัตรา PDR รอบแรกได้ 100%
3. ในระยะ 300 เมตร สภาวะมีสิ่งกีดขวาง (NLOS): สัญญาณวิทยุถดถอยลงไปถึง -107.62 dBm ทำให้ PDR ในรอบแรกหล่นลงมาเหลือ 55% (เกิด Packet Loss 45%) แต่ด้วยประสิทธิภาพของโปรโตคอล ARQ ฝั่งรับสามารถส่งคำสั่งขอส่งซ่อมเฉพาะ 45 ชิ้นส่วนที่ขาดหายไป จนกระทั่งรวบรวมข้อมูลได้ครบถ้วนและประกอบกลับเป็นไฟล์รูปภาพ received_image.jpg ที่สมบูรณ์ได้สำเร็จทุกครั้ง

5.2 ข้อเสนอแนะ (Recommendations)

จากการดำเนินโครงงานและสรุปผลการทดสอบระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกลผ่านคลื่นวิทยุ LoRa (LoRa Serial Bridge) คณะผู้จัดทำได้พบประเด็นที่สามารถนำไปปรับปรุงและพัฒนาต่อยอดเพื่อเพิ่มประสิทธิภาพ ความเร็ว และความน่าเชื่อถือของระบบในอนาคต โดยแบ่งข้อเสนอแนะออกเป็นด้านต่างๆ ดังนี้:

5.2.1 ด้านการปรับปรุงโปรโตคอลการสื่อสารและซอฟต์แวร์

- การปรับขนาดแพ็กเกจแบบพลวัต (Dynamic Chunk Size Allocation): ปัจจุบันระบบฟิกซ์ขนาดชิ้นส่วนข้อมูลไว้ที่ 100 Bytes เพื่อความปลอดภัยของชุดคำสั่ง AT ในอนาคตควรพัฒนารูปแบบ Adaptive Chunking ที่สามารถปรับขนาด Chunk ตามค่า RSSI/SNR ในขณะนั้น เช่น เพิ่มขนาดเป็น 150–200 Bytes เมื่อสัญญาณดี เพื่อลดค่า Overload จาก Header และลดระยะเวลาการส่งรวม
- การปรับปรุงอัลกอริทึมการบีบอัดภาพ (Advanced Image Compression): ควรทดสอบนำรูปแบบไฟล์ภาพที่มีอัตราการบีบอัดสูงกว่า JPEG เช่น WebP หรือ AVIF มาใช้งาน เพื่อลดขนาดไฟล์ไบนารีลงอีก 20–30% ในระดับความคมชัดที่เท่าเดิม ช่วยให้ใช้เวลาในการส่งผ่านแบนด์วิดท์ LoRa น้อยลงอย่างมีนัยสำคัญ
- การบูรณาการระบบแก้ไขข้อผิดพลาดไปข้างหน้า (Forward Error Correction - FEC): นอกเหนือจากการพึ่งพาโปรโตคอล ARQ (Re-transmission) เพียงอย่างเดียว การเพิ่ม Reed-Solomon Code หรือ parity bits เข้าไปในระดับซอฟต์แวร์จะช่วยให้ฝั่งรับซ่อมแซมแพ็กเกจที่เสียหายเล็กน้อยได้เองโดยไม่ต้องยิงร้องขอส่งซ่อม ลดการจราจรหนาแน่นในช่องสัญญาณวิทยุ

5.2.2 ด้านฮาร์ดแวร์และการแพร่กระจายคลื่นวิทยุ

- การใช้อุปกรณ์สายอากาศที่มีกำลังขยายสูง (High-Gain Antenna Implementation): การทดสอบในระยะ 300 เมตร สภาวะ NLOS พบว่าค่า RSSI ตกลงไปถึง -107.62 dBm ข้อเสนอแนะคือควรเปลี่ยนจากสายอากาศ Rubber Duck เดิม ไปใช้สายอากาศแบบ Omni-directional ที่มีกำลังขยายสูงขึ้น (เช่น 5 dBi – 8 dBi) หรือสายอากาศ Directional Yagi ในจุดติดตั้งคงที่ เพื่อเพิ่ม Link Margin และเสถียรภาพสัญญาณ
- การปรับค่าพารามิเตอร์ LoRa แบบอัตโนมัติ (Adaptive Data Rate - ADR): ควรพัฒนาให้โหนดสื่อสารสามารถตกลงปรับเปลี่ยนค่า Spreading Factor (SF) และ Bandwidth (BW) ร่วมกันตามสภาพแวดล้อมได้ เช่น ปรับลดลงมาใช้ SF7 ในระยะใกล้เพื่อเพิ่ม Bitrate และขยับไปใช้ SF10–SF12 เมื่ออยู่ระยะไกลหรือถูกกีดขวางสูง

5.2.3 ด้านการนำไปประยุกต์ใช้งานจริง

- การประยุกต์ใช้ในระบบเฝ้าระวังและ IoT (Remote Surveillance & IoT Integration): สามารถนำโครงงานนี้ไปต่อยอดร่วมกับกล้อง ESP32-CAM หรือเซนเซอร์ตรวจจับเพื่อส่งภาพนิ่งถ่ายโอนจากพื้นที่ห่างไกลที่ไม่มีสัญญาณเซลลูลาร์ (Cellular Dead Zones) เช่น พื้นที่เกษตรกรรมแม่นยำ (Smart Agriculture), การตรวจวัดป่าไม้ หรือระบบเตือนภัยภัยพิบัติ
- การรักษาความปลอดภัยของข้อมูล (Data Security Integration): เพื่อให้พร้อมสำหรับการใช้งานในเชิงพาณิชย์หรือข้อมูลสำคัญ ควรเพิ่มการเข้ารหัสข้อมูลก่อนการแปลง Base64 เช่น การใช้ AES-128 หรือ AES-256 เพื่อป้องกันการดักจับข้อมูล (Eavesdropping) กลางอากาศผ่านคลื่นวิทยุไร้สาย

5.3 แนวทางการพัฒนาต่อยอด

เพื่อยกระดับโครงงานระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกลผ่านคลื่นวิทยุ LoRa (LoRa Serial Bridge) จากระบบต้นแบบ (Prototype) ไปสู่การเป็นระบบที่สมบูรณ์และรองรับการใช้งานจริงในระดับอุตสาหกรรมหรือภาคสนาม คณะผู้จัดทำได้วางแนวทางการพัฒนาต่อยอดในอนาคตออกเป็น 4 ด้านหลัก ดังนี้:

5.3.1 การพัฒนาเป็นระบบโหนดเครือข่ายไร้สาย

- การขยายโครงข่ายแบบ Mesh: พัฒนาโปรโตคอลการรับ-ส่งข้อมูลให้รองรับโครงสร้างเครือข่ายแบบ Mesh Network เพื่อให้แต่ละโหนด (Node) สามารถทำหน้าที่เป็นตัวทวนสัญญาณ (Repeater) ช่วยส่งผ่านข้อมูลต่อกันเป็นทอดๆ ได้ ซึ่งจะช่วยข้ามผ่านสิ่งกีดขวางในสภาวะ NLOS และขยายระยะทางการส่งข้อมูลภาพได้ไกลขึ้นอย่างไม่จำกัดระยะสายตา
- การจัดการโหนดแบบ Multi-Point: ปรับเปลี่ยนโปรโตคอลจากการสื่อสารแบบจุดต่อจุด (Point-to-Point) ให้รองรับการเชื่อมต่อแบบ Point-to-Multipoint โดยมีโหนดศูนย์กลาง (Gateway) รับภาพ

จากโหนดลูกข่ายหลายๆ จุดพร้อมกัน พร้อมระบบจัดการลำดับคิวข้อมูล (Queue Management) เพื่อป้องกันการชนกันของสัญญาณ (Data Collision)

5.3.2 การย้ายประมวลผลไปยัง Edge Computing และฮาร์ดแวร์ฝังตัว

- การทำงานแบบ Standalone โดยไม่พึ่งพา PC: พัฒนาระบบให้สามารถทำงานได้จบในตัวเองในระดับ Edge Device โดยนำบอร์ดประมวลผลขนาดเล็ก เช่น Raspberry Pi หรือใช้โมดูลกล้อง ESP32-CAM ร่วมกับไมโครคอนโทรลเลอร์ มาทำหน้าที่ถ่ายภาพ บีบอัดไฟล์ภาพ ซอยย่อยข้อมูล (Chunking) และสั่งงานผ่าน LoRa ได้โดยตรง โดยไม่ต้องเชื่อมต่อกับคอมพิวเตอร์ A และ B
- การตรวจจับภาพด้วยปัญญาประดิษฐ์ที่โหนดปลายทาง (TinyML Integration): ประยุกต์ใช้โมเดล Machine Learning ขนาดเล็กบนโหนดฝั่งส่ง เพื่อวิเคราะห์รูปภาพก่อนส่ง เช่น ส่งเฉพาะภาพที่มีการตรวจพบวัตถุ คน หรือสิ่งผิดปกติ (Event-Triggered Transmission) แทนการส่งภาพตามระยะเวลา ซึ่งจะช่วยประหยัดพลังงานและแบนด์วิดท์ของระบบได้อย่างมาก

5.3.3 การบริหารจัดการพลังงานและการประยุกต์ใช้พลังงานทางเลือก

- การรองรับระบบพลังงานโซลาร์เซลล์ (Solar Powered Nodes): ออกแบบวงจรจัดการพลังงาน (Power Management Unit) ร่วมกับแผงโซลาร์เซลล์และแบตเตอรี่ LiFePO4 / Li-ion เพื่อให้โหนดสื่อสารฝั่งส่งสามารถติดตั้งและทำงานในพื้นที่ห่างไกลได้อย่างต่อเนื่องโดยไม่ต้องเปลี่ยนแบตเตอรี่
- การใช้โหมดประหยัดพลังงานขั้นสูง (Deep Sleep Integration): ปรับแต่งเฟิร์มแวร์บน ESP32 และสั่งการโมดูล RYLR998 ให้เข้าสู่โหมดประหยัดพลังงาน (Deep Sleep / Low Power Mode) ในช่วงเวลาที่ไม่มีการส่งข้อมูล และกำหนดเวลาตื่น (Wake-up Timer) หรือตื่นตามการสั่งการของเซนเซอร์ เพื่อลดการใช้พลังงานสะสม

5.3.4 การพัฒนาส่วนประสานผู้ใช้และการเชื่อมต่อระบบคลาวด์

- การพัฒนา Web Application และ Mobile App: พัฒนาส่วนประสานผู้ใช้ (Dashboard) ในรูปแบบ Web Application หรือแอปพลิเคชันบนมือถือ เพื่อแสดงผลสถานะการรับ-ส่งภาพ ค่าความแรงสัญญาณ (RSSI/SNR), เปอร์เซ็นต์ Packet Loss และแสดงรูปภาพที่รับได้แบบ Real-time
- การเชื่อมต่อกับระบบคลาวด์และ IoT Protocols: พัฒนาให้โหนดฝั่งรับ (Gateway) สามารถส่งต่อไฟล์รูปภาพและข้อมูลการสื่อสารขึ้นไปยัง Cloud Storage (เช่น AWS S3, Google Cloud) หรือส่งผ่านโปรโตคอล MQTT / Firebase เพื่อให้ผู้ใช้งานสามารถเข้าถึงภาพถ่ายจากระยะไกลผ่านอินเทอร์เน็ตได้ทุกที่ทุกเวลา

บรรณานุกรม

[1]การจัดทำบรรณานุกรมตามมาตรฐาน APA 7th Edition สำหรับโครงงานระบบรับ-ส่งไฟล์รูปภาพไร้สายระยะไกลผ่านคลื่นวิทยุ LoRa (LoRa Serial Bridge) โดยใช้ ESP32 และโมดูล REYAX RYLR998:

[2]กิตติพงษ์ สมบูรณ์. (2565). *การพัฒนาระบบสื่อสารไร้สายระยะไกลด้วยเทคโนโลยี LoRa สำหรับงานไอโอที*. สำนักพิมพ์มหาวิทยาลัยเทคโนโลยีมหานคร.

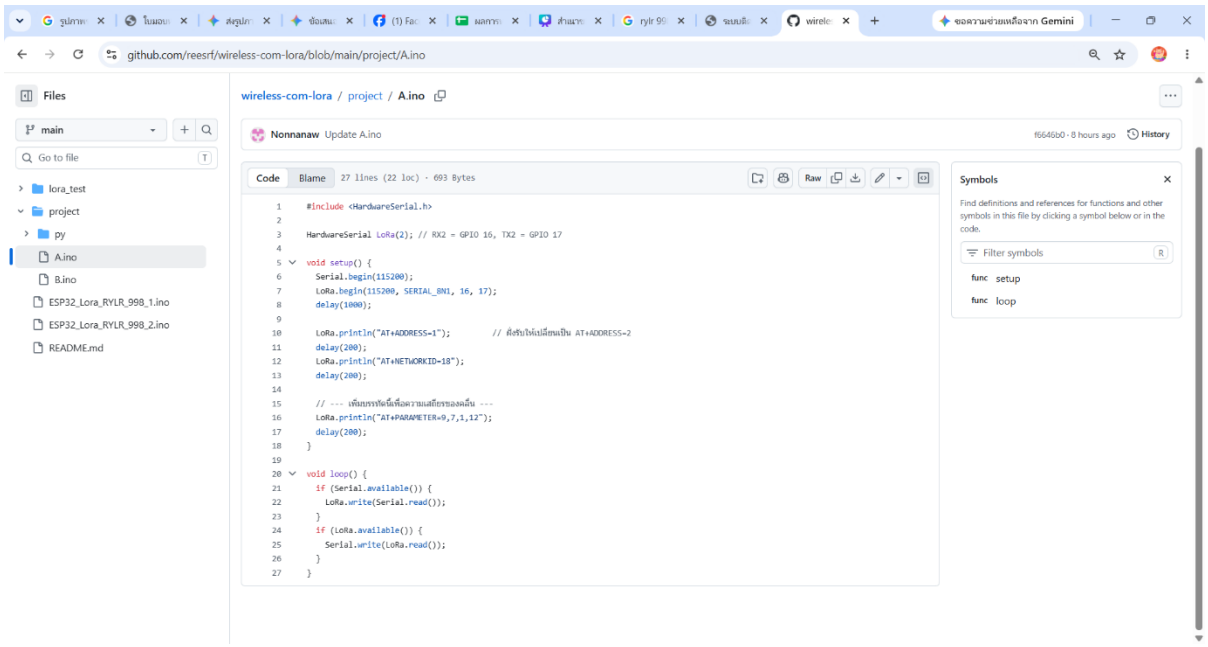
[3]วิชัย ปัญญาไว. (2564). *ไมโครคอนโทรลเลอร์ ESP32 และการประยุกต์ใช้งานในระบบอินเทอร์เน็ตของสรรพสิ่ง (IoT)*. กรุงเทพฯ: ซีเอ็ดยูเคชั่น.

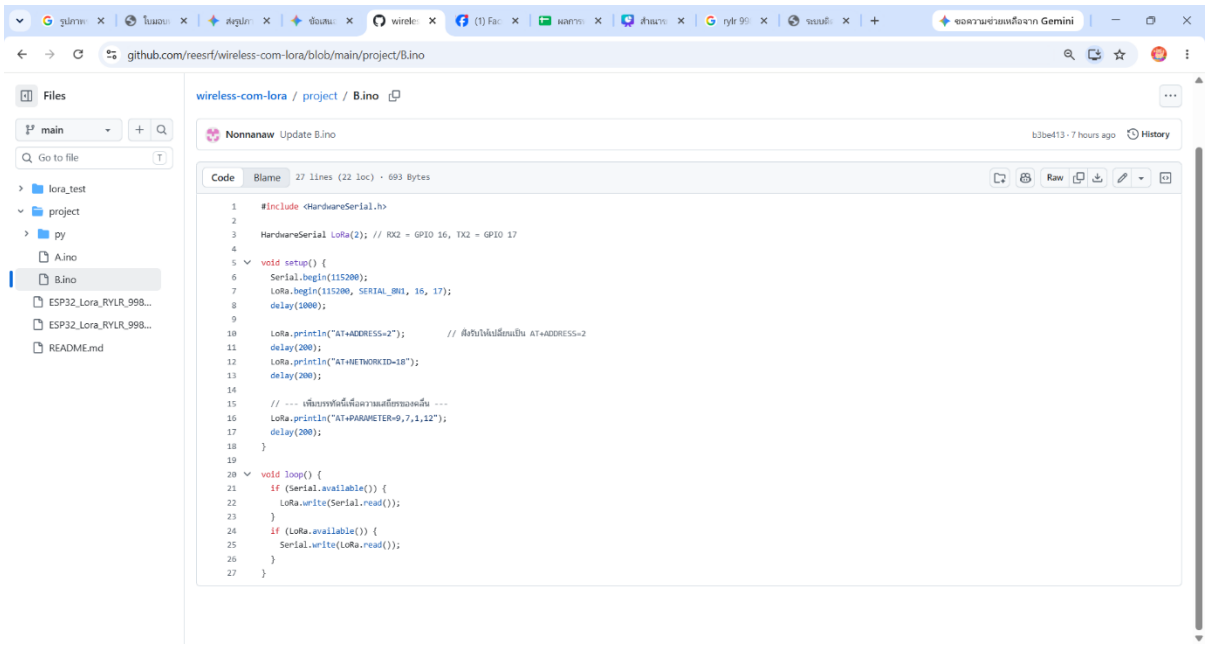
[4]อนันต์ สุขเจริญ. (2566). *เทคโนโลยีการสื่อสารไร้สาย ย่านความถี่ AS923 และการประยุกต์ใช้ในประเทศไทย*. *วารสารวิศวกรรมศาสตร์และเทคโนโลยี*, 15(2), 45–58.

[5]Augusto, P., & Santos, R. (2021). *Data transmission techniques over LoRa physical layer for image transfer in remote areas*. *IEEE Internet of Things Journal*, 8(14), 11230–11242. https://doi.org/10.1109/JIOT.2021.3061234

ภาคผนวก ก

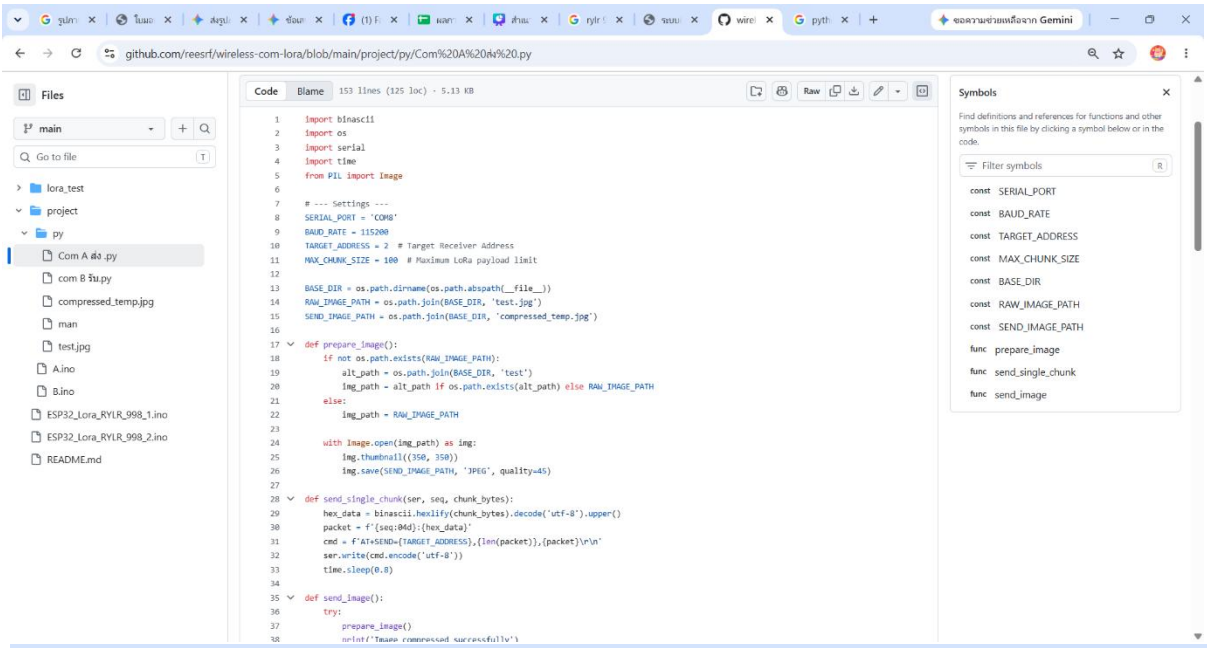
ซอร์สโค้ดเฟิร์มแวร์ ESP32


github.com/reesrf/wireless-com-lora/blob/main/project/A.ino
Files
main
Go to file
lora_test
project
py
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
wireless-com-lora / project / A.ino
Nonnanaw Update A.ino
f6646b0 · 8 hours ago
History
Code
Blame
27 lines (22 loc) · 693 Bytes
Raw
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
func setup
func loop
#include <HardwareSerial.h>
HardwareSerial LoRa(2); // RX2 = GPIO 16, TX2 = GPIO 17
void setup() {
Serial.begin(115200);
LoRa.begin(115200, SERIAL_8N1, 16, 17);
delay(1000);
LoRa.println("AT+ADDRESS=1"); // ตั้งรับให้เปลี่ยนเป็น AT+ADDRESS=2
delay(200);
LoRa.println("AT+NETWORKID=18");
delay(200);
// --- เพิ่มบรรทัดนี้เพื่อความเสถียรของคลื่น ---
LoRa.println("AT+PARAMETER=9,7,1,12");
delay(200);
}
void loop() {
if (Serial.available()) {
LoRa.write(Serial.read());
}
if (LoRa.available()) {
Serial.write(LoRa.read());
}
}


github.com/reesrf/wireless-com-lora/blob/main/project/B.ino
Files
main
Go to file
lora_test
project
py
A.ino
B.ino
ESP32_Lora_RYLR_998...
ESP32_Lora_RYLR_998...
README.md
wireless-com-lora / project / B.ino
Nonnanaw Update B.ino
b3be413 · 7 hours ago
History
Code
Blame
27 lines (22 loc) · 693 Bytes
Raw
#include <HardwareSerial.h>
HardwareSerial LoRa(2); // RX2 = GPIO 16, TX2 = GPIO 17
void setup() {
Serial.begin(115200);
LoRa.begin(115200, SERIAL_8N1, 16, 17);
delay(1000);
LoRa.println("AT+ADDRESS=2"); // ตั้งรับให้เปลี่ยนเป็น AT+ADDRESS=2
delay(200);
LoRa.println("AT+NETWORKID=18");
delay(200);
// --- เพิ่มบรรทัดนี้เพื่อความเสถียรของคลื่น ---
LoRa.println("AT+PARAMETER=9,7,1,12");
delay(200);
}
void loop() {
if (Serial.available()) {
LoRa.write(Serial.read());
}
if (LoRa.available()) {
Serial.write(LoRa.read());
}
}

ภาคผนวก ข

ซอร์สโค้ดเฟิร์มแวร์ python



github.com/reesrf/wireless-com-lora/blob/main/project/py/Com%20A%20ส่ง%20.py

Files

main

Go to file

- lora_test
- project
  - py
    - Com A ส่ง .py
    - com B รับ.py
    - compressed_temp.jpg
    - man
    - test.jpg
  - A.ino
  - B.ino
  - ESP32_Lora_RYLR_998_1.ino
  - ESP32_Lora_RYLR_998_2.ino
  - README.md

Code | Blame | 153 lines (125 loc) · 5.13 KB | Raw

```python
import binascii
import os
import serial
import time
from PIL import Image

# --- Settings ---
SERIAL_PORT = 'COM8'
BAUD_RATE = 115200
TARGET_ADDRESS = 2  # Target Receiver Address
MAX_CHUNK_SIZE = 100  # Maximum LoRa payload limit

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
RAW_IMAGE_PATH = os.path.join(BASE_DIR, 'test.jpg')
SEND_IMAGE_PATH = os.path.join(BASE_DIR, 'compressed_temp.jpg')

def prepare_image():
    if not os.path.exists(RAW_IMAGE_PATH):
        alt_path = os.path.join(BASE_DIR, 'test')
        img_path = alt_path if os.path.exists(alt_path) else RAW_IMAGE_PATH
    else:
        img_path = RAW_IMAGE_PATH

    with Image.open(img_path) as img:
        img.thumbnail((350, 350))
        img.save(SEND_IMAGE_PATH, 'JPEG', quality=45)

def send_single_chunk(ser, seq, chunk_bytes):
    hex_data = binascii.hexlify(chunk_bytes).decode('utf-8').upper()
    packet = f'{seq:04d}:{hex_data}'
    cmd = f'AT+SEND={TARGET_ADDRESS},{len(packet)},{packet}\r\n'
    ser.write(cmd.encode('utf-8'))
    time.sleep(0.8)

def send_image():
    try:
        prepare_image()
        print('Image compressed successfully')
    except Exception as e:
        print(f'Failed to prepare image: {e}')
        return

    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=3)
        time.sleep(2)
    except Exception as e:
        print(f'Failed to open Serial port: {e}')
        return

    with open(SEND_IMAGE_PATH, 'rb') as f:
        image_bytes = f.read()

    total_bytes = len(image_bytes)

    # --- Ensure at least 100 packets ---
    total_chunks = (total_bytes + MAX_CHUNK_SIZE - 1) // MAX_CHUNK_SIZE

    if total_chunks < 100:
        total_chunks = 100

    actual_chunk_size = (total_bytes + total_chunks - 1) // total_chunks

    print(f'Actual file size: {total_bytes} bytes')
    print(f'Divided into: {total_chunks} chunks (max {actual_chunk_size} bytes each)')

    ser.reset_input_buffer()

    # 1. Send START signal
    start_payload = f'START:{total_bytes}:{total_chunks}'
    cmd = f'AT+SEND={TARGET_ADDRESS},{len(start_payload)},{start_payload}\r\n'
    ser.write(cmd.encode('utf-8'))
    print('Sending START signal...')
    time.sleep(1.5)

    # 2. Split and store all chunks
    chunks_dict = {}
    for i in range(total_chunks):
        chunk = image_bytes[i * actual_chunk_size : (i + 1) * actual_chunk_size]
        chunks_dict[i] = chunk

    # 3. First round of transmission
    for i in range(total_chunks):
        print(f'Sending Chunk {i+1}/{total_chunks} [Seq {i:04d}]...')
        send_single_chunk(ser, i, chunks_dict[i])

    # Variable for retransmit statistics
    total_retransmits = 0
    max_retries = 10

    # 4. Loop to verify completeness
    for retry in range(max_retries):
        print(f'\n[Verification Round {retry+1}] Sending CHECK command...')
        check_payload = 'CHECK'
        cmd = f'AT+SEND={TARGET_ADDRESS},{len(check_payload)},{check_payload}\r\n'

        ser.reset_input_buffer()
        ser.write(cmd.encode('utf-8'))

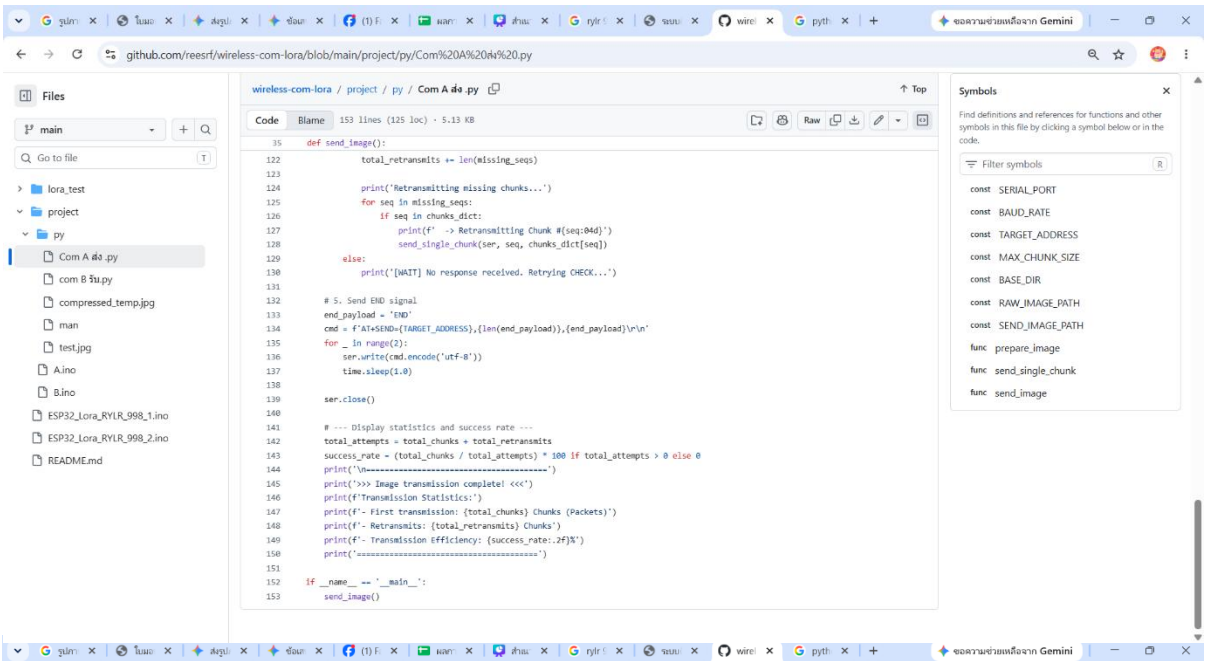
        status = None
        start_time = time.time()
        while time.time() - start_time < 5.0:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if line.startswith('+RCV='):
                parts = line.split(',')
```

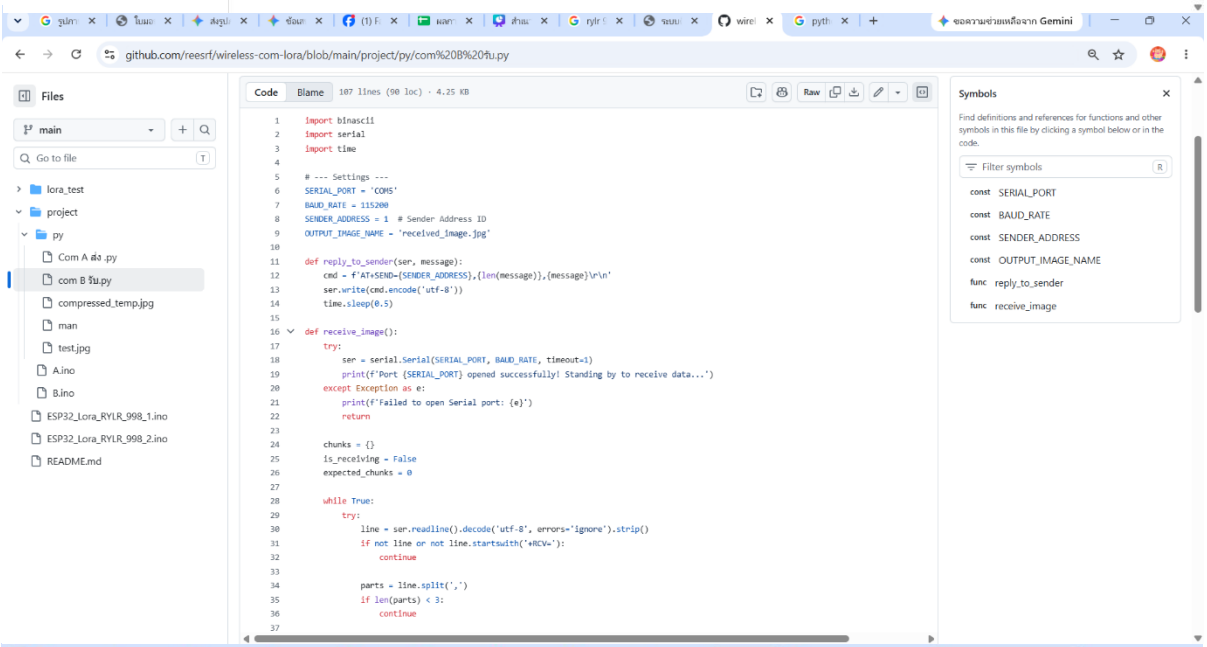
Symbols

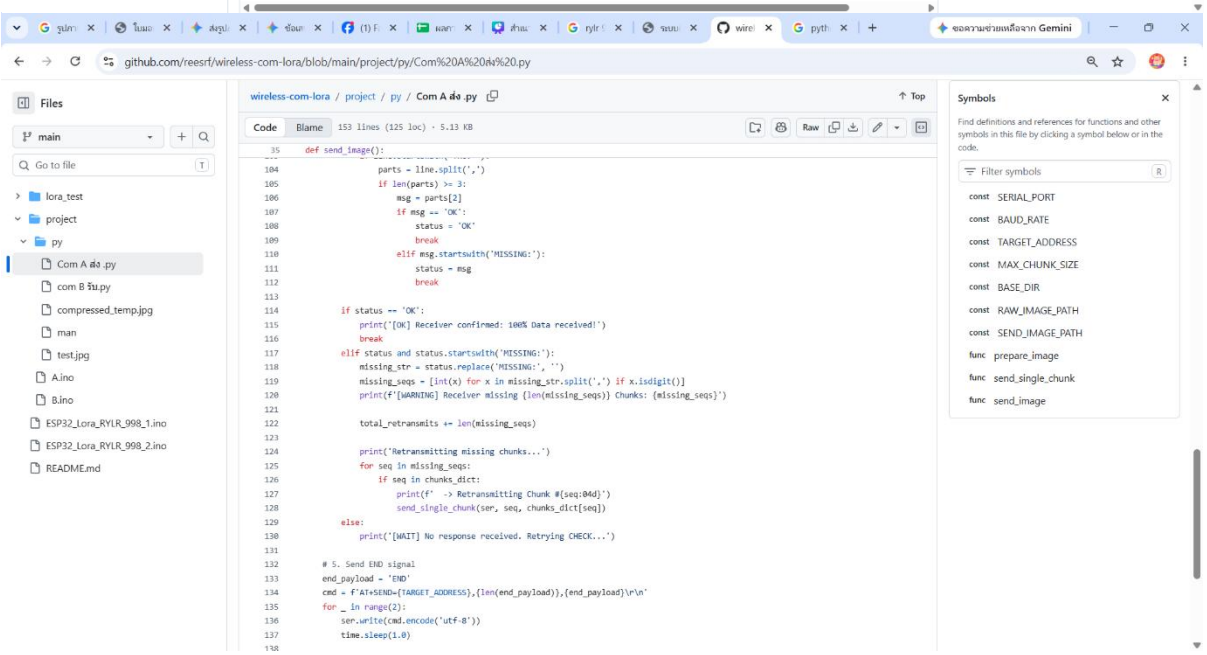
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

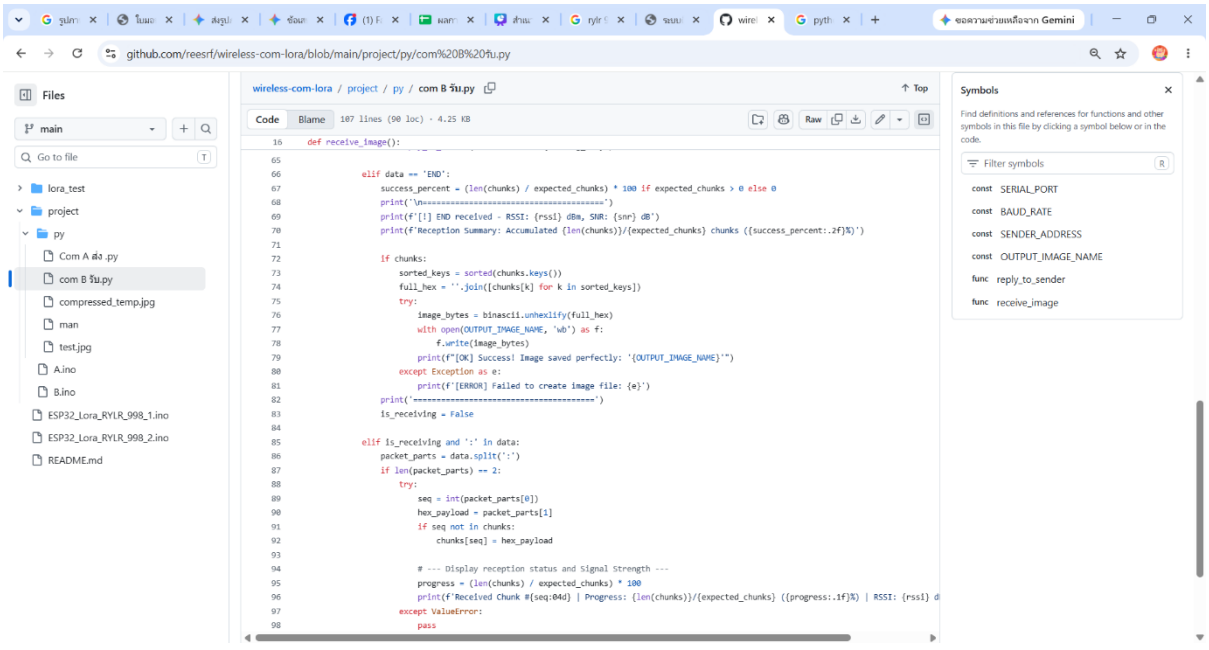
Filter symbols

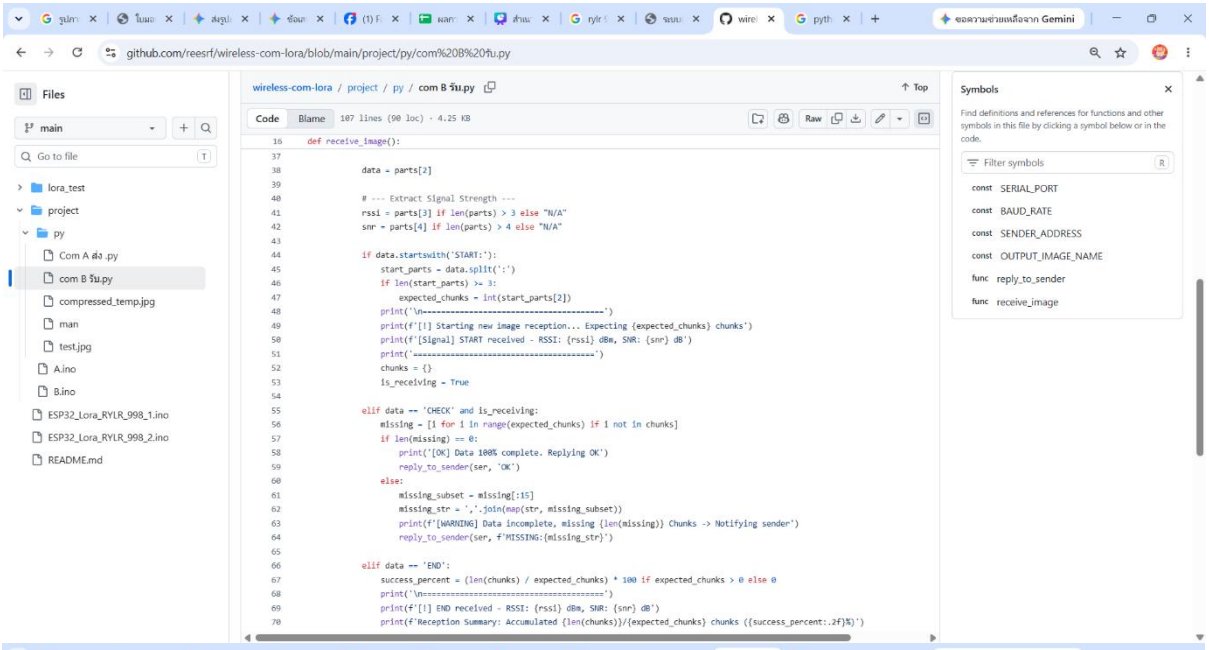
- const SERIAL_PORT
- const BAUD_RATE
- const TARGET_ADDRESS
- const MAX_CHUNK_SIZE
- const BASE_DIR
- const RAW_IMAGE_PATH
- const SEND_IMAGE_PATH
- func prepare_image
- func send_single_chunk
- func send_image

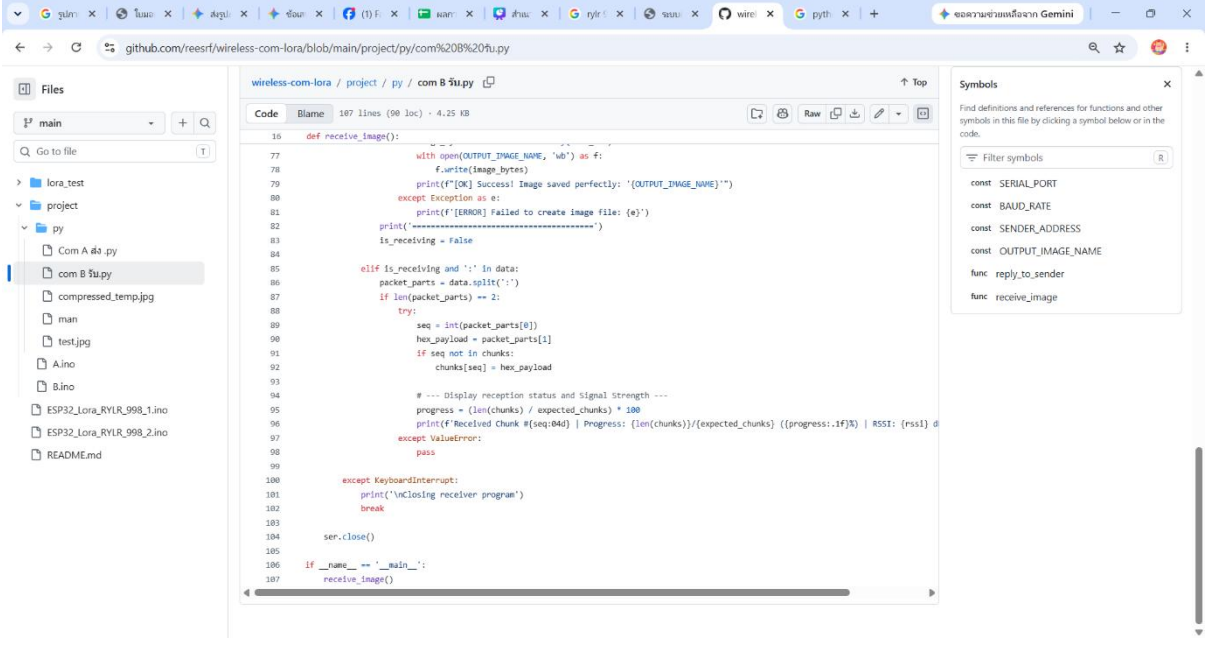

github.com/reesrf/wireless-com-lora/blob/main/project/py/Com%20A%20ส่ง%20.py
Files
main
Go to file
lora_test
project
py
Com A ส่ง .py
com B รับ.py
compressed_temp.jpg
man
test.jpg
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
wireless-com-lora / project / py / Com A ส่ง .py
Code
Blame
153 lines (125 loc) · 5.13 KB
Raw
Top
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
const SERIAL_PORT
const BAUD_RATE
const TARGET_ADDRESS
const MAX_CHUNK_SIZE
const BASE_DIR
const RAW_IMAGE_PATH
const SEND_IMAGE_PATH
func prepare_image
func send_single_chunk
func send_image


github.com/reesrf/wireless-com-lora/blob/main/project/py/com%20B%20รับ.py
Files
main
Go to file
lora_test
project
py
Com A ส่ง .py
com B รับ.py
compressed_temp.jpg
man
test.jpg
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
Code
Blame
107 lines (90 loc) · 4.25 KB
Raw
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
const SERIAL_PORT
const BAUD_RATE
const SENDER_ADDRESS
const OUTPUT_IMAGE_NAME
func reply_to_sender
func receive_image


github.com/reesrf/wireless-com-lora/blob/main/project/py/Com%20A%20ส่ง%20.py
Files
main
Go to file
lora_test
project
py
Com A ส่ง .py
com B รับ.py
compressed_temp.jpg
man
test.jpg
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
wireless-com-lora / project / py / Com A ส่ง .py
Code
Blame
153 lines (125 loc) · 5.13 KB
Raw
Top
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
const SERIAL_PORT
const BAUD_RATE
const TARGET_ADDRESS
const MAX_CHUNK_SIZE
const BASE_DIR
const RAW_IMAGE_PATH
const SEND_IMAGE_PATH
func prepare_image
func send_single_chunk
func send_image


github.com/reesrf/wireless-com-lora/blob/main/project/py/com%20B%20รับ.py
Files
main
Go to file
lora_test
project
py
Com A ส่ง .py
com B รับ.py
compressed_temp.jpg
man
test.jpg
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
wireless-com-lora / project / py / com B รับ.py
Code
Blame
107 lines (90 loc) · 4.25 KB
Raw
Top
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
const SERIAL_PORT
const BAUD_RATE
const SENDER_ADDRESS
const OUTPUT_IMAGE_NAME
func reply_to_sender
func receive_image


github.com/reesrf/wireless-com-lora/blob/main/project/py/com%20B%20รับ.py
Files
main
Go to file
lora_test
project
py
Com A ส่ง .py
com B รับ.py
compressed_temp.jpg
man
test.jpg
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
wireless-com-lora / project / py / com B รับ.py
Code
Blame
107 lines (90 loc) · 4.25 KB
Raw
Top
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
const SERIAL_PORT
const BAUD_RATE
const SENDER_ADDRESS
const OUTPUT_IMAGE_NAME
func reply_to_sender
func receive_image


github.com/reesrf/wireless-com-lora/blob/main/project/py/com%20B%20รับ.py
Files
main
Go to file
lora_test
project
py
Com A ส่ง .py
com B รับ.py
compressed_temp.jpg
man
test.jpg
A.ino
B.ino
ESP32_Lora_RYLR_998_1.ino
ESP32_Lora_RYLR_998_2.ino
README.md
wireless-com-lora / project / py / com B รับ.py
Code
Blame
107 lines (90 loc) · 4.25 KB
Raw
Top
Symbols
Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.
Filter symbols
const SERIAL_PORT
const BAUD_RATE
const SENDER_ADDRESS
const OUTPUT_IMAGE_NAME
func reply_to_sender
func receive_image